

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 316 — Past Questions & Materials
by Renance

Course: CSC 316
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 3
Levels: 300

This study pack was compiled and restructured by Renance for CSC 316 students at Federal University of Petroleum Resources, Effurun. It brings together 3 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 3 — CSC 316 computer architecture and organization 2024/2025

Level 300 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



FEDERAL UNIVERSITY OF PETROLEUM RESOURCES EFFURUN
DEPARTMENT OF COMPUTER SCIENCE
1st SEMESTER UNDERGRADUATE EXAMINATION (2024/2025 SESSION)
CSC316: COMPUTER ARCHITECTURE AND ORGANIZATION (3 UNITS)

INSTRUCTIONS:

1. Answer Four (4) Questions Only.
2. Time allowed: 2½ hours.
3. Answer Each Question on a New Page.

Question 1.

- a) A computer is a complex system that contains millions of elementary electronic components. How then can one clearly describe them? (6mks).
- b) In general terms, when does the two's complement operation on an n-bit integer produce the same integer? (6mks).
- c) An address field in an instruction contains decimal value 14. Where is the corresponding operand located for: (i) immediate addressing (ii) Direct addressing (iii) indirect addressing. (6mks).

Question 2.

- a) Convert the expression $A + B - C$ to postfix notation using Dijkstra's algorithm. Show the steps involved. Is the result equivalent to $(A + B) - C$ or $A + (B - C)$? (12mks).
- b) List and briefly explain two common ways of generating the condition to be tested in a conditional branch instruction. (4mks).
- c) At the integrated circuit level, what are the three principal constituents of a computer system. (3mks).

Question 3.

- a) The raw speed of the microprocessor will not achieve its potential unless it is fed a constant stream of work to do in the form of computer instructions. Briefly explain the techniques built into contemporary processors to accomplish this. (6mks).
- b) Briefly explain the various methods of accessing units of data as it affects memory types. (6mks).
- c) What are the basic elements of floating addition and subtraction? (6mks).

Question 4.

- a) What are the advantages and disadvantages of using a variable-length instruction? (6mks).
- b) Justify the assertion that a 32-bit instruction is probably much less than twice as useful as a 16-bit instruction. (6mks).
- c) Using instruction cycle state diagram for execution of instructions, explain the elements of a machine instruction (6mks).

Question 5.

- a) What facts go into determining the use of the addressing bits of an instruction. (6mks).
- b) Show how the following floating-point calculations are performed (where significands are truncated to 4 decimal digits) (i) $(0.2255 * 10^2) * (0.1234 * 10^2)$ (ii) $(0.8833 * 10^2) * (0.5555 * 10^5)$. (8mks).
- c) If an instruction contains four addresses, what might be the purpose of each address. (4mks).

Paper 2 of 3 — csc 316 introduction to computer Architecture

Level 300 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.

CSC 316

**INTRODUCTION TO
COMPUTER ARCHITECTURE**

CHAPTER ONE INTRODUCTION

1.0 Definition

Computer architecture refers to those parameters of a computer system that are visible to a programmer or those parameters that have a direct impact on the logical execution of a program. Examples of architectural attributes include the instruction set, the number of bits used to represent different data types, I/O mechanisms, and techniques for addressing memory.

Computer organization refers to the operational units and their interconnections that realize the architectural specifications. Examples of organizational attributes include those hardware details transparent to the programmer, such as control signals, interfaces between the computer and peripherals, and the memory technology used.

In this course we will touch upon all those factors and finally come up with the concept how these attributes contribute to build a complete computer system.

As an example, it is an architectural design issue whether a computer will have a multiple instruction. It is an organizational issue whether that instruction will be implemented by a special multiply unit or by the method of repeated addition by using the add unit of the system.

1.2 Computer Organization and Architecture

Computer technology has made incredible improvement in the past half century. In the early part of computer evolution, there were no stored-program computer, the computational power was less and on the top of it the size of the computer was a very huge one.

Today, a personal computer has more computational power, more main memory, more disk storage, smaller in size and it is available in affordable cost. This rapid rate of improvement has come both from advances in the technology used to build computers and from innovation in computer design.

The task that the computer designer handles is a complex one: Determine what attributes are important for a new machine, then design a machine to maximize performance while staying within cost constraints. This task has many aspects, including instruction set design, functional organization, logic design, and implementation.

1.3 Basic Computer Model and different units of Computer [10m]

The model of a computer can be described by four basic units in high level abstraction. These basic units are:

- Central Processor Unit
- Input Unit

- Output Unit
- Memory Unit

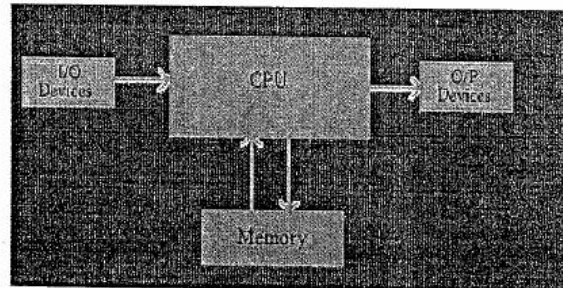


Figure 1: Basic Computer Model and different units of Computer

1.3.1 Central Processor Unit [CPU]:

Central processor unit consists of two basic blocks:

- The program control unit has a set of registers and control circuit to generate control signals.
- The execution unit or data processing unit contains a set of registers for storing data and an Arithmetic and Logic Unit (ALU) for execution of arithmetic and logical operations.

In addition, CPU may have some additional registers for temporary storage of data.

1.3.2 Input Unit:

With the help of input unit, data from outside can be supplied to the computer. Program or data is read into main storage from input device or secondary storage under the control of CPU input instruction. Example of input devices: Keyboard, Mouse, Hard disk, Floppy disk, CD-ROM drive, etc.

1.3.3 Output Unit:

With the help of output unit computer results can be provided to the user or it can be stored in storage device permanently for future use. Output data from main storage go to output device under the control of CPU output instructions.

Example of output devices: Printer, Monitor, Plotter, Hard Disk, Floppy Disk etc.

1.3.4 Memory Unit:

Memory unit is used to store the data and program. CPU can work with the information stored in memory unit. This memory unit is termed as primary memory or main memory module. These are basically semiconductor memories.

There are two types of semiconductor memories -

- Volatile Memory: RAM (Random Access Memory).
- Non-Volatile Memory : ROM (Read only Memory), PROM (Programmable ROM)

EPROM (Erasable PROM), EEPROM (Electrically Erasable PROM).

1.3.4.1 Secondary Memory:

There is another kind of storage device, apart from primary or main memory, which is known as secondary memory. Secondary memories are non-volatile memory and it is used for permanent storage of data and program. Example of secondary memories:

Hard Disk, Floppy Disk, Magnetic Tape ----- These are magnetic devices,
CD-ROM ----- is optical device

Thumb drive (or pen drive) ----- is semiconductor memory.

1.4 Basic Working Principle of a Computer

Before going into the details of working principle of a computer, we will analyse how computer works with the help of a small hypothetical computer. In this small computer, we do not consider about Input and Output unit. We will consider only CPU and memory module. Assume that somehow we have stored the program and data into main memory. We will see how CPU can perform the job depending on the program stored in main memory.

1.4.1 Consider the Arithmetic and Logic Unit (ALU) of Central Processing Unit:

An ALU can perform four arithmetic operations and four logical operations

To distinguish between arithmetic and logical operation, we may use a signal line,

0 - in that signal, represents an arithmetic operation and

1 - in that signal, represents a logical operation.

In the similar manner, we need another two signal lines to distinguish between four arithmetic operations.

The different operations and their binary code is as follows:

Arithmetic		Logical	
000	ADD	100	OR
001	SUB	101	AND
010	MULT	110	NAND

MDR - memory data register
MAR - memory access register } Registers

011	DIV	111	ADD
-----	-----	-----	-----

The control unit's task is to generate the appropriate signal at right moment. There is an instruction decoder in CPU which decodes this information in such a way that computer can perform the desired task. The simple model for the decoder may be considered that there is three input lines to the decoder and correspondingly it generates eight output lines. Depending on input combination only one of the output signals will be generated and it is used to indicate the corresponding operation of ALU.

In our simple model, we use three storage units in CPU,

Two -- for storing the operand and
One -- for storing the results. } Register

These storage units are known as register.

But in computer, we need more storage space for proper functioning of the Computer. Some of them are inside CPU, which are known as register. Other bigger junk of storage space is known as primary memory or main memory. The CPU can work with the information available in main memory only. To access the data from memory, we need two special registers one is known as Memory Data Register (MDR) and the second one is Memory Address Register (MAR).

Data and program is stored in main memory. While executing a program, CPU brings instruction and data from main memory, performs the tasks as per the instruction fetch from the memory. After completion of operation, CPU stores the result back into the memory.

1.5 Designing for Performance

Year by year, the cost of computer systems continues to drop dramatically, while the performance and capacity of those systems continue to rise equally dramatically.

1.5.1 Microprocessor Speed

What gives the Pentium or the PowerPC such mind-boggling power is the relentless pursuit of speed by processor chip manufacturers. In microprocessor, the addition of new circuits, and the speed boost that comes from reducing the distance between them, has improved performance four or five fold every three years.

But the raw speed of the microprocessor will not achieve its potential unless it is fed a constant stream of work to do in the form of computer instructions. Anything that gets in the way of that smooth flow undermines the power of the processor. Accordingly, while the chipmakers have

been busy learning how to fabricate chips of greater and greater density, the processor designers must come up with ever more elaborate techniques for feeding the monster. Among the techniques built into contemporary processors are the following:

Techniques built into contemporary processors

- Branch prediction: the processor looks ahead in the instruction code fetched from memory and predicts which branches or groups of instructions are likely to be processed next. If the processor guesses right most of the time, it can pre-fetch the correct instruction and buffer them so that the processor is kept busy. The more sophisticated examples of this strategy predict not just the next branch but multiple branches ahead. Thus, branch prediction increases the amount of work available for the processor to execute.
- Data flow analysis: the processor analyzes which instructions are dependent on each other's results or data to create an optimized schedule of instructions. In fact, instructions are scheduled to be executed when ready, independent of the original program order. This prevents unnecessary delay.
- Speculative execution: using branch prediction and data analysis, some processors speculatively execute instruction ahead of their actual appearance in the program execution, holding the results in temporary locations. This enables the processor to keep its execution engines as busy as possible by executing instructions that are likely to be needed.

This is fetch more
is made a used to
Send data to the
processor ports
cannot handle

These and other sophisticated techniques are made necessary by the sheer power of the processor. They make it possible to exploit the raw speed of the processor.

1.5.2 Performance Balance

While processor power has raced ahead at breakneck speed, other critical components of the computer have not kept up. The result is a need to look for performance balance: an adjusting of the organization and architecture to compensate for the mismatch among the capabilities of the various components.

Nowhere is the problem created by such mismatch more critical than in the interface between processor and main memory. While processor speed and memory capacity has grown rapidly, the speed with which data can be transferred between main memory and the processor has lagged badly. The interface between processor and main memory is the most crucial pathway in the entire computer, because it is responsible for carrying a constant flow of program instructions and data between memory chips and the processor. If memory or the pathway fails.

to keep pace with the processor's insistent demands, the processor stalls in a wait state, and valuable processing time is lost.

There are a number of ways that a system architect can attack this problem, all of which are reflected in contemporary computer designs. Examples include the followings:

- Increase the number of bits that are retrieved at one time by making DRAMs "wider" rather "deeper" and by using wide data paths.
- Change the DRAM interface to make it more efficient by including a cache or other buffering scheme on the DRAM chip.
- Reduce the frequency of memory access by incorporating increasingly complex and efficient cache structures between the processor and main memory. This include the incorporation of one or more cache on the processor chip as well as on the off-chip cache close to the processor chip.
- Increase the interconnect bandwidth between processors and memory by using higher speed buses and by using a hierarchy of buses to buffer and structure data flow.

1.6 Pentium and PowerPC Evolution

The two major contemporary examples of computer families are the Intel Pentium and the PowerPC. The Pentium represents the results of decades of design efforts on complex instruction set computers (CISC). It incorporates the sophisticated design principles once found only on mainframes and supercomputers and serves as an excellent example of CISC design. The PowerPC is a direct descendant of the first RISC system, the IBM801 is one of the most powerful and best-designed RISC-based system in the market.

1.6.1. Pentium

In terms of market share, Intel has ranked as the number one market of microprocessors for decades, a position it seems unlikely to yield. The evolution of its flagship microprocessor product serves as a good indicator of the evolution of computer technology in general.

Below is are highlights of the evolution of the Intel product line:

- 8080: the world's first general-purpose microprocessor. This was an 8-bit machine, with an 8-bit data path to memory. The 8080 was used in the first personal computer, the Altair.
- 8086: a far more powerful 16bit machine. In addition to a wider data path and larger registers, the 8086 sported an instruction cache, or queue, that pre-fetches a few instruction before they are executed.

- 80286: this extension of the 8086 enabled addressing a 16-Mbyte memory instead of just 1 Mbyte.
- 80386: Intel's first 32-bit machine, and a major overhaul of the product. With a 32-bit architecture, the 80386 rivaled the complexity and power of minicomputers and mainframes introduced just a few years earlier. This was the first Intel processor to support multitasking, meaning it could run multiple programs at the same time.
- 80486: the 80486 introduced the use of much sophisticated and powerful cache technology and sophisticated pipelining. The 80486 also offered a built-in math coprocessor, offloading complex math operations from the main CPU.
- Pentium: with the Pentium, Intel introduced the use of superscalar techniques, which allow multiple instructions to execute in parallel.
- Pentium Pro: the Pentium Pro continued the move into superscalar organization begun the Pentium, with aggressive use of register renaming branch prediction, data flow analysis and speculative execution.
- Pentium II: the Pentium II incorporated Intel MMX technology, which is designed specifically to process video, audio, and graphics data efficiently.
- Pentium III: the Pentium III incorporates additional floating point instructions to support 3D graphics software
- Pentium 4: the Pentium 4 includes additional floating point and other enhancement for multimedia
- Itanium: the new generation of Intel processor makes use of a 64-bit organization with the IA-64 architecture.

1.6.2 PowerPC

IBM entered into an alliance with Motorola, developer of 68000 series of microprocessors, and Apple, which used the Motorola chip in its Macintosh computers. The result is a series of machines that implement the PowerPC architecture. This architecture is derived from the POWER architecture. Changes were made to add key missing features and to enable more efficient implementation of eliminating some instructions and relaxing the specification to eliminate some troublesome special cases. The resulting PowerPC architecture is a superscalar RISC system.

The following are the principal members of the PowerPC family:

- 601: the purpose of the 601 was to bring the PowerPC architecture to the marketplace as quickly as possible. The 601 is a 32-bit machine.

- 603: intended for low-end desktop and portable computers. It is also a 32-bit machine, comparable in performance with the 601, but with lower cost and a more efficient implementation.
- 604: intended for desktop computers and low-end servers. Again, this is a 32-bit machine, but it uses much more sophisticated advanced superscalar design techniques to achieve greater performance
- 620: intended for high-end servers. The first member of the PowerPC family to implement a full 64-bit architecture, including 64-bit registers and data paths.
- 740/750: also known as the G3 processor. The processor integrates two levels of cache in the main processor chip, providing significant performance improvement over a comparable machine with off-chip cache organization.
- G4: this processor increases the parallelism and internal speed of the processor chip.

Evolution of Intel Product Line are:

8080
8086
80286
80386
80486
Pentium
Pentium Pro
Pentium II
Pentium III
Pentium 4
Itanium

Principle members of the powerPc family

*601
*603
*604
*620
*740/750
*G4

An overflow flag is set to 1 if the result of a computation exceeds the length of the register into which it is stored.

CHAPTER TWO

* COMPUTER ARITHMETIC

2.0 Introduction

The two principal concerns for computer arithmetic are two ways in which numbers are represented (binary format) and the algorithm used for the basic arithmetic operations (add, subtract, multiply, divide). These two considerations apply both to integer and floating-point arithmetic.

2.1 The Arithmetic and Logic Unit (ALU)

The ALU is that part of the computer that actually performs arithmetic and logical operations on data. All of the other elements of the computer system – control unit, registers, memory, I/O – are there mainly to bring data into the ALU for it to process and then take the results back out.

An ALU and, indeed, all electronic components in the computer are based on the use of simple digital logic devices that can store binary digits and perform simple Boolean logic operations.

Figure 2 indicates, in general terms, how the ALU is interconnected with the rest of the processor.

Data are presented to the ALU in registers, and the results of an operation are stored in registers. These registers are temporary storage locations within the processor that are connected by signal paths to the ALU. The ALU may also set flags as the result of an operation. For example, an overflow flag is set to 1 if the result of a computation exceeds the length of the register into which it is stored. The flag values are also stored in registers within the processor. The control unit provides signals that control the operation of the ALU and the movement of the data into and out of the ALU.

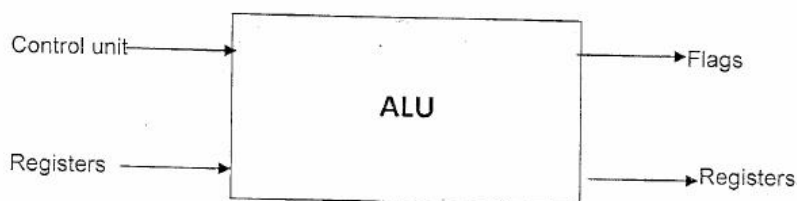


Figure 2: ALU inputs and outputs

2.2 Binary Number System

We have already mentioned that computer can handle two type of signals, therefore, to represent any information in computer, we have to take help of these two signals. These two signals corresponds to two levels of electrical signals, and symbolically we represent them as 0 and 1.

In our day to day activities for arithmetic, we use the Decimal Number System. The decimal number system is said to be of base, or radix (10) because it uses ten digits and the coefficients are multiplied by power of 10. A decimal number such as 5273 represents a quantity equal to 5 thousands plus 2 hundreds, plus 7 tens, plus 3 units. The thousands, hundreds, etc. are powers of 10 implied by the position of the coefficients. To be more precise, 5273 should be written as:

$$5 \times 10^3 + 2 \times 10^2 + 7 \times 10^1 + 3 \times 10^0$$

However, the convention is to write only the coefficient and from their position deduce the necessary power of 10. In decimal number system, we need 10 different symbols. But in computer we have provision to represent only two symbols. So directly we cannot use decimal number system in computer arithmetic.

For computer arithmetic we use binary number system. The binary number system uses two symbols to represent the number and these two symbols are 0 and 1. The binary number system is said to be of base 2 or radix 2, because it uses two digits and the coefficients are multiplied by power of 2.

The binary number 110011 represents the quantity equal to:

$$1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 51 \text{ (in decimal)}$$

We can use binary number system for computer arithmetic.

2.3 Integer Representation

Any integer can be stored in computer in binary form. As for example: The binary equivalent of integer 107 is 1101011, so 1101011 are stored to represent 107.

* What is the size of Integer that can be stored in a Computer? *

It depends on the word size of the computer. If we are working with 8-bit computer, then we can use only 8 bits to represent the number. The eight bit computer means the storage organization for data is 8 bits. In case of 8-bit numbers, the minimum number that can be stored in computer is 00000000 (0) and maximum number is 11111111 (255) (if we are working with natural numbers).

2 | 107
2 | 53 r 1
2 | 26 r 1
2 | 13 r 0
2 | 6 r 1
2 | 3 r 0
2 | 1 r 1
0 r 1

1101011

10

So, the domain of number is restricted by the storage capacity of the computer. Also it is related to number system; above range is for natural numbers. In general, for n-bit number, the range for natural number is from 0 to $2^n - 1$.

Any arithmetic operation can be performed with the help of binary number system. Consider the following two examples, where decimal and binary additions are shown side by side.

01101000	104
00110001	49
10011001	153

In the above example, the result is an 8-bit number, as it can be stored in the 8-bit computer, so we get the correct results.

10000001	129
10101010	178

msb ← 100101011 307

In the above example, the result is a 9-bit number, but we can store only 8 bits, and the most significant bit (msb) cannot be stored. The result of this addition will be stored as (00101011) which is 43 and it is not the desired result. Since we cannot store the complete result of an operation, and it is known as the overflow case.

The smallest unit of information is known as BIT (Binary digit).

The binary number 110011 consists of 6 bits and it represents:
 $1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 32 + 16 + 2 + 1 = 51_{10}$

2.3.1 Octal Number: The octal number system is said to be of base, or radix 8, because it uses 8 digits and the coefficients are multiplied by power of 8. Eight digits used in octal system are: 0, 1, 2, 3, 4, 5, 6 and 7.

2.3.2 Hexadecimal number: The hexadecimal number system is said to be of base, or radix 16, because it uses 16 symbols and the coefficients are multiplied by power of 16.

Sixteen digits used in hexadecimal system are: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E and F.

Consider the following addition example:

Decimal = 10 bits
 Octal = 8 bits
 Hexa decimal = 16 bits
 Binary = 2 bits

Binary	Octal	Hexadecimal	Decimal
01101000	150	68	104
00111010	072	3A	58
-----	-----	-----	-----
10100010	242	A2	162

0 - positive
1 - negative

2.4 Signed Integer

We know that for n-bit number, the range for natural number is from 0 to $2^n - 1$. For n-bit, we have all together 2^n different combination, and we use these different combination to represent 2^n numbers, which ranges from 0 to $2^n - 1$.

If we want to include the negative number, naturally, the range will decrease. Half of the combinations are used for positive number and other half is used for negative number.

For n-bit representation, the range is from -2^{n-1} to $2^{n-1} - 1$.

For example, if we consider 8-bit number, then range for natural number is from 0 to 255; but for signed integer the range is from -127 to +127.

2.4.1 Representation of signed integer

We know that for n-bit number, the range for natural number is from 0 to $2^n - 1$.

There are three different schemes to represent negative number:

- Signed-Magnitude form.
- 1's complement form.
- 2's complement form.

Schemes for representing Signed Integer.

2.4.2 Signed magnitude form:

In signed-magnitude form, one particular bit is used to indicate the sign of the number, whether it is a positive number or a negative number. Other bits are used to represent the magnitude of the number. For an n-bit number, one bit is used to indicate the signed information and remaining (n-1) bits are used to represent the magnitude. Therefore, the range is from -2^{n-1} to $2^{n-1} - 1$.

Generally, Most Significant Bit (MSB) is used to indicate the sign and it is termed as signed bit. 0 in signed bit indicates positive number and 1 in signed bit indicates negative number.

For example, 01011001 represents +169 and 11011001 represents -169.

There are several drawbacks to sign-magnitude representation. One is that addition and subtraction require a consideration of both the signs of the numbers and their relative magnitudes to carry out the required operations.

- drawbacks to sign-magnitude representation are:
- addition & subtraction require a consideration of both the signs of the numbers & their relative magnitude.
 - There are two representation of 0. i.e. $+0 = 00000000$ and $-0 = 10000000$

*599*2#

*902#

Another drawback is that there are two representation of 0.

$+0 = 00000000$

$-0 = 10000000$ (sign magnitude) ?

This is inconvenient because it is slightly more difficult to test for 0 (an operation performed frequently on computers) than if there were a single representation. Because of these drawbacks, sign-magnitude representation is rarely used in implementing the integer portion of the ALU. Instead, the most common scheme is two's complement representation.

2.4.3. The concept of 2's complement

The concept of ^{2's} complements is used to represent signed number. Like sign-magnitude, 2's complement representation uses the most significant bit as a sign bit, making it easy to test whether an integer is positive or negative. It differs from the use of sign-magnitude representation in the way that the other bits are interpreted. The table below shows the characteristics of 2's complement representation and arithmetic.

1. Range	-2^{n-1} through $2^{n-1}-1$
2. No of representation of 0	One (1)
3. Negation	Take the Boolean complement of each bit of the corresponding +ve number, then add 1 to the resulting bit pattern viewed as an unsigned integers
4. Expansion of bit length	Add additional bit positions to the left and fill in with the value of the original sign bit
5. Overflow rule	If two numbers with the same sign (both +ve or both -ve) are added, then overflow occurs if and only if the result has the opposite sign
6. Subtraction rule	To subtract B from A, take the 2's complement of B and add it to A

Note that the 1's complement form of any binary number is obtained simply by changing each 0 in the number to a 1 and each 1 to a 0. In 1's complement form, the sign bit is made as 1 and the magnitude is converted from true magnitude form to its 1's complement. For example, the number -52 would be represented as follows:

-52 = 1 0110100 (true-magnitude form)
 1 1001011 (1's complement form)

$$\begin{array}{r}
 2 \overline{) 17} \\
 2 \overline{) 34} \\
 2 \overline{) 68} \\
 \hline
 0010011100 \\
 11000.0
 \end{array}$$

Note that the sign bit is not complemented but is kept as a 1 to indicate a negative number. You should verify that $-7.25 = 111000.10_2$ in 1's complement.

On the other hand the 2's complement form of a binary number is formed by taking its 1's complement and adding 1 to the least significant bit position. The procedure is illustrated below for converting 001100100 (decimal 52) to its 2's complement form.

$$\begin{array}{r}
 00110100 \xrightarrow{\text{LSB}} \\
 11001011 \quad \text{1's complement} \\
 + \quad 1 \quad \text{-- add 1 to LSB to form 2's complement} \\
 \hline
 11001100
 \end{array}$$

Example:

Add +9 and -4

$$\text{Ans: } +9 = 00001001$$

$$-4 = 11111100$$

$$100000101$$

Carry discarded

Sign bit

$$\begin{array}{r}
 2 \overline{) 4} \\
 2 \overline{) 2} \\
 2 \overline{) 1} \\
 \hline
 001
 \end{array}$$

$$\begin{array}{r}
 2 \overline{) 4} \\
 2 \overline{) 2} \\
 2 \overline{) 1} \\
 \hline
 001
 \end{array}$$

100000100
1111011
1111100

Note that -4 will be in its 2's complement form. Thus, +4 (0000100) must be converted to -4 (11111100).

Consider a number system of base- r or radix- r . There are two types of complements,

- The radix complement or the r 's complement.
- The diminished radix complement or the $(r-1)$'s complement.

2.4.3.1 Diminished Radix Complement:

Given a number N in base r having n digits, the $(r-1)$'s complement of N is defined as $(r^n - 1) - N$.

For decimal numbers, $r = 10$ and $r-1 = 9$, so the 9's complement of N is $(10^n - 1) - N$.

e.g., 9's complement of 5642 is $9999 - 5642 = 4357$.

2.4.3.2 Radix Complement:

The r 's complement of an n -digit number in base r is defined as $(r^n - N)$ for $N \neq 0$ and 0 for $N = 0$. r 's complement is obtained by adding 1 to the $(r-1)$'s complement,

$$\text{since } (r^n - N) = [(r^n - 1) - N] + 1$$

e.g., 10's complement of 5642 is 9's complement of 5642 + 1, i.e., $4357 + 1 = 4358$

e.g., 2's complement of 1010 is 1's complement of 1010 + 1, i.e., $0101 + 1 = 0110$.

$$(r^n - 1) - N$$

water tutorial on 316.

2.4.4. Representation of Signed integer in 2's complement form:

Consider the eight bit number 01011100, 2's complements of this number is 10100100. If we perform the following addition:

$$\begin{array}{r}
 01011100 \\
 10100011 \\
 \hline
 10000000
 \end{array}$$

MSB

Since we are considering an eight bit number, so the 9th bit (MSB) of the result cannot be stored. Therefore, the final result is 00000000.

Since the addition of two number is 0, so one can be treated as the negative of the other number. So, 2's complement can be used to represent negative number.

Decimal 2's Complement 1's complement Signed Magnitude

+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	----	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	-----	-----

41
 2 0000
 2 0000
 2 5000
 2 2000
 2 1000
 2 0000

↑ 101001

2.5 Representation of Real Number

Binary representation of 41.6875 is 101001.1011

Therefore any real number can be converted to binary number system. There are two schemes to represent real number:

- Fixed-point representation
- Floating-point representation

2.5.1 Fixed-point representation:

Binary representation of 41.6875 is 101001.1011. To store this number, we have to store two information,

- the part before decimal point and
- the part after decimal point.

This is known as fixed-point representation where the position of decimal point is fixed and number of bits before and after decimal point are also predefined.

If we use 16 bits before decimal point and 8 bits after decimal point, in signed magnitude form, the range is $-2^{16} - 1$ to $+2^{16} - 1$ and the precision is 2^{-8} .

One bit is required for sign information, so the total size of the number is 25 bits (1(sign) + 16(before decimal point) + 8(after decimal point)).

2.5.2 Floating-point representation:

In this representation, numbers are represented by a mantissa comprising the significant digits and an exponent part of Radix R. The format is: $\text{mantissa} * R^{\text{exponent}}$

Numbers are often normalized, such that the decimal point is placed to the right of the first non-zero digit. For example, the decimal number,

5236 is equivalent to $5.236 * 10^3$

To store this number in floating point representation, we store 5236 in mantissa part and 3 in exponent part.

2.5.2.1 IEEE standard floating point format:

We can represent a number in the form:

$$\pm S * B^{\pm E}$$

This number can be stored in a binary word with three fields:

- sign: plus or minus
- significand: S
- exponent: E

The base B is implicit and need not be stored because it is the same for all numbers. Typically, it is assumed that the radix point is to the right of the leftmost, or the MSB of the significand. That is, there is one bit to the left of the radix point.

2.5.2.2 Floating Point Arithmetic

For addition and subtraction it is very necessary to ensure that both operands have the same exponent value. This may require shifting the radix point on one of the operands to achieve alignment. Multiplication and division are more straight-forward. The table below summarizes the basic operations for floating-point arithmetic.

Floating-point number	Arithmetic operations
$X = X_s * B^{X_E}$	$X + Y = (X_s * B^{X_E - Y_E} + Y_s) * B^{Y_E}$ $X - Y = (X_s * B^{X_E - Y_E} - Y_s) * B^{Y_E}$
$Y = Y_s * B^{Y_E}$	$X * Y = (X_s * Y_s) * B^{X_E + Y_E}$ $X / Y = (X_s / Y_s) * B^{X_E - Y_E}$

Example:

Perform the four arithmetic operations for the following X and Y numbers:

$$X = 0.3 * 10^2$$

$$Y = 0.2 * 10^3$$

Ans:

$$X + Y = (0.3 * 10^{2-3} + 0.2) * 10^3 = 0.23 * 10^3 = 230$$

$$X - Y = (0.3 * 10^{2-3} - 0.2) * 10^3 = (-0.17) * 10^3 = -170$$

$$X * Y = (0.3 * 0.2) * 10^{2+3} = 0.06 * 10^5 = 6000$$

$$X / Y = (0.3 / 0.2) * 10^{2-3} = 1.5 * 10^{-1} = 0.15$$

Handwritten notes:

$$X_E = -16$$

$$B$$

$$X_E = 2$$

$$10^3$$

$$X_s = 0.3$$

$$Y_s = 0.2$$

A floating point operation may provide one of these conditions:

- Conditions for Floating Point:
- Exponent overflow: a +ve exponent exceeds the maximum possible exponent value. In some systems this may be designated as +∞ or -∞
 - Exponent underflow: a -ve exponent is less than the minimum possible exponent (e.g. -200 is less than -127). This means that the number is too small to be represented and it may be reported as 0
 - Significand underflow: in the process of aligning significands, digits may flow off the right end of the significand

Subtrahend: A number or quantity to be subtracted from another.

- Significand overflow: the addition of two significands of the same sign may result in a carry out of the most significant bit. This can be fixed by re-alignment

2.5.2.3 Algorithm for Addition and Subtraction in Floating-Point Arithmetic

In floating-point arithmetic, addition and subtraction are more complex than multiplication and division. This is because of the need for alignment. There are four basic phases of the algorithm for addition and subtraction

Phase 1 - zero check: because addition and subtraction are identical except for a sign change, the process begins by changing the sign of the subtrahend if it is a subtract operation. Next, if either operand is 0, the other is reported as the result.

Phase 2 - significand alignment: the next phase is to manipulate the numbers so that the two exponents are equal. Alignment may be achieved by shifting either the smaller number to the right (increasing the component) or shifting the large number to the left. Because either operation may result in the loss of digits, it is the smaller number that is shifted; any digits that are lost are therefore relatively small significance. The alignment is achieved by repeatedly shifting the magnitude portion of the significand right 1 digit and incrementing the exponent until the two exponents are equal.

Phase 3 - addition: next, the two significands are added together, taking into account their signs. Because the sign may differ, the result may be 0. There is also the possibility of significand overflow by 1 digit. If so, the significand of the result is shifted right and the exponent is incremented. An exponent overflow could occur as a result; this would be reported and the operation failed

Phase 4 - normalization: the final phase normalizes the result. Normalization consists of shifting significand digits left until the most significant digit is 1. Each shift causes a decrement of the exponent and this could cause an exponent underflow. Finally, the result must be rounded off and then reported.

To see the need for aligning exponents, consider the following decimal addition:

$$(123 * 10^0) + (456 * 10^{-2})$$

Clearly we cannot just add the significands. The digits must first be set into equivalent positions that is, the 4 of the second number must be aligned with the 3 of the first. Under these conditions, the two exponents will be equal, which is the mathematical condition under which two numbers in this form can be added. Thus,

$$(123 * 10^0) + (456 * 10^{-2}) = (123 * 10^0) + (4.56 * 10^0) = 127.56 * 10^0.$$

2.5.3 Representation of Character

Since we are working with 0's and 1's only, to represent character in computer we use strings of 0's and 1's only. To represent character we are using some coding scheme, which is nothing but a mapping function. Some of standard coding schemes are: ASCII, EBCDIC, UNICODE.

ASCII : American Standard Code for Information Interchange. It uses a 7-bit code. All together we have 128 combinations of 7 bits and we can represent 128 character.

As for example 65 = 1000001 represents character 'A'.

EBCDIC : Extended Binary Coded Decimal Interchange Code. It uses 8-bit code and we can represent 256 character.

UNICODE : It is used to capture most of the languages of the world. It uses 16-bit. Unicode provides a unique number for every character, no matter what the platform, no matter what the program, no matter what the language. The Unicode Standard has been adopted by such industry leaders as Apple, HP, IBM, JustSystem, Microsoft, Oracle, SAP, Sun, Sybase, Unisys and many others.

2.6 Addition and Subtraction in Integer Representation

In twos complement notation, the negation of an integer can be formed with the following rules:

- Take the Boolean complement of each bit of the integer (including the sign bit). That is set each 1 to 0 and each 0 to 1
- Treating the result as an unsigned integer, add 1

The two-step process is referred to as the twos complement operation, or taking of the twos complement of an integer.

Addition in twos complement is illustrated below:

1001 = -7	1100 = -4	0011 = 3	1100 = -4
+0101 = 5	+ 0100 = 4	0100 = 4	1111 = -1
1110 = -2	10000 0	0111 = 7	11011 = -5

The above illustrate successful operations. If the result of the operation is positive, we get a positive number in ordinary binary notation. If the result of the operation is negative, we get a negative number in twos complement form. Note that in some instances, there is a carry bit beyond the end of the word, which is ignored.

On any addition, the result may be larger than can be held in the word size being used. This condition is called overflow. When overflow occurs, the ALU must signal this fact so that no attempt is made to use the result. To detect overflow, the following rule is observed. If two numbers are added, and they are both positive or both negative, then overflow occurs if and

$$\begin{array}{r} 01 \\ 0010 \\ 1111 \\ 0111 \\ 23 \end{array}$$

$$\begin{array}{r} 11 \\ \times 11 \\ \hline 11 \\ 11 \\ \hline 1001 \end{array}$$

{ Subtrahend
minuend }

$$\begin{array}{r} 2 \overline{) 9} \\ 2 \overline{) 41} \\ 2 \overline{) 20} \\ 2 \overline{) 10} \\ 0 \end{array}$$

only if the result has opposite sign. Note that an overflow to occur whether or not there is a carry.

Subtraction is also easily handled with the following rule: to subtract one number (subtrahend) from another (minuend), take the two's complement (negation) of the subtrahend and add it to the minuend. Thus subtraction is achieved using addition.

0010 = 2	0101 = 5	1011 = -5	0101 = 5
1001 = -7	1110 = -2	1110 = -2	0010 = 2
1011 = -5	10011 = 3	11001 = -7	0111 = 7

2.7 Multiplication

Compared with addition and subtraction, multiplication is a complex operation, whether performed in hardware or software. A wide variety of algorithms have been used in various computers.

When multiplying unsigned integers, some important observations can be made:

- Multiplication involves the generation of partial products, one for each digit in the multiplier. These partial products are then summed up to produce the final product
- The partial products are easily defined. When the multiplier bit is 0, the partial product is 0. When the multiplier is 1, the partial product is the multiplicand
- The total product is produced by summing the partial products. For this operation, each successive partial product is shifted one position to the left relative to the preceding partial product
- The multiplication of two n-digit binary integers is a product of up to 2n bits in length (e.g. $11 \times 11 = 1001$)

One of the algorithm used in two's complement multiplication is the Booth's algorithm can be illustrated as with this example: multiply 2 by -4 (base 10 numbers). Note the multiplicand is 2 and the multiplier is -4.

- The upper half of the product register will initialize with the value of zero (0)
- The lower half of the product register will be initialized with the value of the multiplier 2 in this example, and will by the last repetition of these steps, contain the correct value for the product of the multiplication.

We will keep up with 5 values during this multiplication demonstration

1. Product multiplier
2. Multiplicand
3. Current bit (least most)

AP21

4. Previous bit (least most)

5. Operation

We will or will not perform an "operation" based on the current and previous bits.

00 do nothing

01 add multiplicand to upper (left) half of the product register

10 subtract multiplicand to upper (left) half of product register

11 do nothing

From the above example, we will use 5 bits i.e. $2_{10} * -4_{10}$

2 in base 2 is 00010

-4 in base 2 is 11100

-2 in base 2 is 11110

Why do we need the -2? The operation 10 means we need to subtract the multiplicand. Since the multiplicand is positive, it is the easy way to subtract it. We add the complement of the positive of 2. Negative 2 is the complement of positive 2. So if we add the binary value of negative 2 to the product register we are still actually subtracting it since it is a signed integer. This is why I recommend adding the complement of the multiplicand for the subtract operation.

Rep	Product Register	Multiplicand	Current bit	Previous bit	Operation on product register then shift right
0	00000 11100 (-4)	00010 (2)	0	0	Do nothing
1	00000 01110	00010	0	0	Do nothing
2	00000 00111	00010	1	0	Subtract multiplicand from upper product register
2a	11110 00111	New product but still need to shift right keeping the correct "signed" bit			
3	11111 00011	00010	1	1	Do nothing
4	11111 10001	00010	1	1	Do nothing
5	11111 11000	00010	0	1	Add multiplicand to upper product register
5a	00001 11000	Last iteration (#bits=rep) then the answer is in lower half of the register, -8_{10} i.e. 11000			

FILO (First In, Last Out)

322/411/412

2.8 Stacks

A stack is an ordered set of elements, only one of which can be accessed at a time. The point of access is called the top of the stack. The number of elements in the stack is a variable. Items may be added to or deleted from the top of the stack. For this reason, a stack is also known as a pushdown list or last-in-first-out (LIFO) list, First-in-Last-out (FILO).

The figure below shows the basic stack operations. We begin at some point in time when the stack contains some numbers of elements. A PUSH operation ^{add} appends one new item to the top of the stack. A POP operation ^{delete} removes the top item from the stack. In both cases, the top of the stack moves accordingly. Binary operations which require two operands (e.g. multiply, divide, add, subtract) use the top two stack items as operands, pop both items and push the result back onto the stack. Unary operations, which require only one operand (e.g. logical NOT), use the item on the top of the stack.

2.8.1 Stack Implementation

The stack is a useful structure to provide as part of a CPU implementation. One use is to manage procedure calls and returns. Stacks may also be useful to the programmer. An example of this is expression of evaluation.

The implementation of a stack depends in part on its potential uses. If it is desired to make stack operation available to the programmer, then the instruction set will include stack-oriented operations, including PUSH, POP, and operations that use the top one or two stack elements as operands. Because all of these operations refers to a unique location, namely the top of the stack, the address of the operand or operands is implicit and need not be included in the instruction. These are the zero-address instructions.

If the stack mechanism is to be used only by the CPU, for such purpose as procedure handling, then there will not be explicit stack-oriented instructions in the instruction set. In either case, the implementation of a stack requires that there be some set of locations used to store the stack elements. A contiguous block of locations is reserved in main memory (or virtual memory) for the stack. Most of the time, the block is partially filled with stack elements and the remainder is available for stack growth.

Three addresses are needed for proper operation, and these are often stored in CPU register:

- Stack pointer: ^{added} contains the address of the top of the stack. If an item is appended to or deleted from the stack, the pointer is incremented or decremented to contain the address of the new top of the stack.

- Stack base: contains the address of the bottom location in the reserved block. If an attempt is made to POP when the stack is empty, an error is reported.
- Stack limit: contains the address of the other end of the reserved block. If an attempt is made to PUSH when the block is fully utilized for the stack, an error is reported.

2.8.2 Expression Evaluation

Mathematical formulas are usually expressed in what is known as infix notation. In this form, a binary operation appears between the operands (e.g. $a+b$). For complex expressions, parentheses are used to determine the order of evaluation of expressions. For example, $a+(b*c)$ will yield a different result than $(a+b)*c$. To minimize the use of parentheses, operations have an implied precedence. Generally, multiplication takes precedence over addition, so that $a+b*c$ is equivalent to $a+(b*c)$.

An alternative technique is known as reverse Polish or postfix notation, in the notation, the operator follows its two operands. For example,

$a+b$	becomes $ab+$
$a+(b*c)$	becomes $abc*+$
$(a+b)*c$	becomes $ab+c*$

Note that, regardless of the complexity of an expression, no parentheses are required when using reverse Polish.

The advantage of postfix notation is that an expression in this form is easily evaluated using a stack. An expression in postfix notation is scanned from left to right. For each element of the expression, the following rules are applied:

1. If the element is a variable or constant, push it onto the stack
2. If the element is an operator, pop the top two items of the stack, perform the operation, and push the result.

After the entire expression has been scanned, the result is on the top of the stack.

The simplicity of this algorithm makes it a convenient one for evaluating expressions. Accordingly, many compilers will take an expression in a high-level language, convert it to postfix notation, and then generate the machine instruction from the notation. Below is the sequence of machine instructions for evaluating $f = (a-b)/(c+d*e)$ using stack-oriented instructions. Note that, even though the stack-oriented rules were not used in the last two case, the postfix notation served as a guide for generating the machine instructions.

Pop - delete
Push - add

	Stack	General register	Single register
	Push a	Load R1,a	Load d
	Push b	Subtract R1,b	Multiply e
	Subtract	Load R2,d	Add c
	Push c	Multiply R2,e	Store f
	Push d	Add R2,c	Load a
	Push e	Divide R1,R2	Subtract b
	Multiply	Store R1,f	Divide f
	add		Store f
	divide		
	pop f		
Number of instructions	10	7	8
Memory access	10op+ 6d	7op +6d	8op +8d

The process of converting an infix expression to a postfix expression is itself most easily accomplished using a stack. The following algorithm is due to Dijkstra. The infix expressions scanned from left to right and the postfix expression is developed and output during the scan. The steps are as follows:

1. Examine the next element in the input
2. If it is an operand output it (a, b, c, ...)
3. If it is an opening parenthesis, push it onto the stack (()
4. If it is an operator, then (+, -, /, *)
 - If the top of the stack is an opening parenthesis, then push the operator
 - If it has higher priority than the top of the stack (multiply and divide have higher priority than add and subtract), then push the operator
 - Else, pop operation from stack to output, and repeat step 4
5. If it is a closing parenthesis, pop operators to the output until an opening parenthesis is encountered. Pop and discard the opening parenthesis. ()
6. If there is more input, go to step 1
7. If there is no more input, unstack the remaining operands.

Example: using the Dijkstra algorithm convert the expression to a postfix expression:
A+B*C+(D+E)*F

Input	Output	Stack (top on right)
A+B*C+(D+E)*F	Empty	Empty
+B*C+(D+E)*F	A	Empty
B*C+(D+E)*F	A	+
*C+(D+E)*F	AB	+
C+(D+E)*F	AB	+
+(D+E)*F	ABC	+
·(D+E)*F	ABC*+	+
D+E)*F	ABC*+	+
+E)*F	ABC*+D	+
E)*F	ABC*+D	+
) *F	ABC*+DE	+
F	ABC+DE+	+
F	ABC*+DE+	+
Empty	ABC*+DE+F	+
empty	ABC*+DE+F*+	Empty

ASSIGNMENT ONE

1. What is a stored program computer? *A stored program computer is a computer which stores the executing program in memory.*
2. What is the key distinguishing features of a microprocessor?
- ✓ 3. How can you form the negation of an integer in twos complement representation?
- ✓ 4. What is the difference between the twos complement representation of a number and the twos complement of a number?
- ✓ 5. Briefly, explain the following representation: sign magnitude and twos complement.

representation
 #2's complement ~~uses~~ the MSB as a sign bit, making it easy to test whether an integer is positive or negative.

#2's complement ~~form~~ of a binary number is formed by taking its 1's complement and adding 1 to the LSB position.

CHAPTER THREE

INSTRUCTION SET & ADDRESSING

3.1 Machine Instruction

The operation of a CPU is determined by the instruction it executes, referred to as machine instructions or computer instructions. The collection of different instructions is referred to as the instruction set of the CPU.

Each instruction must contain the information required by the CPU for execution. The elements of an instruction are as follows:

3.1.1 Operation Code:

Specifies the operation to be performed (e.g., add, move etc.). The operation is specified by a binary code, known as the operation code or opcode.

3.1.2 Source operand reference:

The operation may involve one or more source operands; that is, operands that are inputs for the operation.

3.1.3 Result operand reference:

The operation may produce a result.

3.1.4 Next instruction reference:

This tells the CPU where to fetch the next instruction after the execution of this instruction is complete:

The next instruction to be fetched is located in main memory. But in case of virtual memory system, it may be either in main memory or secondary memory (disk). In most cases, the next instruction to be fetched immediately follows the current instruction. In those cases, there is no explicit reference to the next instruction. When an explicit reference is needed, then the main memory or virtual memory address must be given.

mnemonics are opcodes represented by an abbreviation
 OR
 An abbreviation used in representing an opcode

Source and result operands can be in one of the three areas:

- main or virtual memory,
- CPU register or
- I/O device.

The steps involved in instruction execution is shown in the figure-

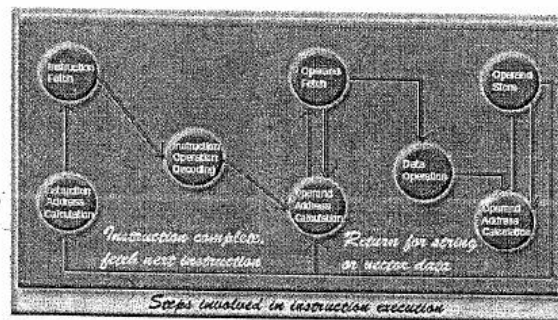


Figure 3.1: instruction cycle state diagram

3.2 Instruction Representation

Within the computer, each instruction is represented by a sequence of bits. The instruction is divided into fields, corresponding to the constituent elements of the instruction. The instruction format is highly machine specific and it mainly depends on the machine architecture. A simple example of an instruction format is shown in the figure 3.2. It is assumed that it is a 16-bit CPU. 4 bits are used to provide the operation code. So, we may have to 16 ($2^4 = 16$) different set of instructions. With each instruction, there are two operands. To specify each operands, 6 bits are used.

It is possible to provide 64 ($2^6 = 64$) different operands for each operand reference. It is difficult to deal with binary representation of machine instructions. Thus, it has become common practice to use a symbolic representation of machine instructions.

Opcodes are represented by abbreviations, called mnemonics that indicate the operations.

Common examples include:

ADD Add
 SUB Subtract
 MULT Multiply
 DIV Division
 LOAD Load data from memory to CPU

STORE Store data to memory from CPU.

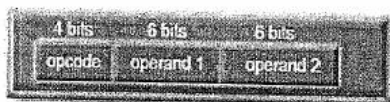


Figure 3.2: A simple instruction format.

Operands are also represented symbolically. For example, the instruction

MULT R, X : R R x X

may mean multiply the value contained in the data location X by the contents of register R and put the result in register R

In this example, X refers to the address of a location in memory and R refers to a particular register. Thus, it is possible to write a machine language program in symbolic form. Each symbolic opcode has a fixed binary representation, and the programmer specifies the location of each symbolic operand.

3.3 Instruction Types

The instruction set of a CPU can be categorized as follows: DP, DS, DM, DC

$R, X : R R \times X$

3.3.1 Data Processing:

Arithmetic and Logic instructions: Arithmetic instructions provide computational capabilities for processing numeric data. Logic (Boolean) instructions operate on the bits of a word as bits rather than as numbers. Logic instructions thus provide capabilities for processing any other type of data. These operations are performed primarily on data in CPU registers.

3.3.2 Data Storage:

Memory instructions: Memory instructions are used for moving data between memory and CPU registers.

3.3.3 Data Movement:

I/O instructions: I/O instructions are needed to transfer program and data into memory from storage device or input device and the results of computation back to the user.

3.3.4 Control:

Test and branch instructions: Test instructions are used to test the value of a data word or the status of a computation. Branch instructions are then used to branch to a different set of instructions depending on the decision made.

test d Status of a computation

3.4 Number of Addresses

Olurun

we need a maximum of two addresses to reference operands

What is the maximum number of addresses one might need in an instruction? Most of the arithmetic and logic operations are either *unary* (one operand) or *binary* (two operands). Thus we need a maximum of two addresses to reference operands. The result of an operation must be stored, suggesting a third address. Finally after completion of an instruction, the next instruction must be fetched, and its address is needed.

This reasoning suggests that an instruction may require to contain *four address references*: two operands, one result, and the address of the next instruction. In practice, *four address instructions are rare*. Most instructions have one, two or three operands addresses, with the address of the next instruction being implicit (obtained from the program counter).

3.5 Instruction Set Design

One of the most interesting, and most analyzed, aspects of computer design is instruction set design. The instruction set defines the functions performed by the CPU. The instruction set is the programmer's means of controlling the CPU. Thus programmer requirements must be considered in designing the instruction set.

Most important and fundamental design issues: **(ODIRA)**

- **Operation repertoire:** How many and which operations to provide, and how complex operations should be.
- **Data Types:** The various type of data upon which operations are performed.
- **Instruction format:** Instruction length (in bits), number of addresses, size of various fields and so on.
- **Registers:** Number of CPU registers that can be referenced by instructions and their use.
- **Addressing:** The mode or modes by which the address of an operand is specified.

These issues are highly inter-related and must be considered together in designing an instruction set

3.6 Types of Operands

Machine instructions operate on data. Data can be categorized as follows :

Addresses: It basically indicates the address of a memory location. Addresses are nothing but the unsigned integer, but treated in a special way to indicate the address of a memory location. Address arithmetic is somewhat different from normal arithmetic and it is related to machine architecture.

Numbers: All machine languages include numeric data types. Numeric data are classified into two broad categories: integer or fixed point and floating point.

Characters: A common form of data is text or character strings. Since computer works with bits, so characters are represented by a sequence of bits. The most commonly used coding scheme is ASCII (American Standard Code for Information Interchange) code.

Logical Data: Normally each word or other addressable unit (byte, half-word, and so on) is treated as a single unit of data. It is sometime useful to consider an n-bit unit as consisting of n 1-bit items of data, each item having the value 0 or 1. When data are viewed this way, they are considered to be logical data. Generally 1 is treated as true and 0 is treated as false.

3.7 Types of Operations

The number of different *opcodes* and their types varies widely from machine to machine. However, some general type of operations are found in most of the machine architecture. Those operations can be categorized as follows: **DALCIST**

- Data Transfer
- Arithmetic
- Logical
- Conversion
- Input Output [I/O]
- System Control
- Transfer Control

3.7.1 Data Transfer:

The most fundamental type of machine instruction is the data transfer instruction. The data transfer instruction must specify several things. First, the location of the source and destination operands must be specified. Each location could be memory, a register, or the top of the stack. Second, the length of data to be transferred must be indicated. Third, as with all instructions with operands, the mode of addressing for each operand must be specified.

The CPU has to perform several task to accomplish a data transfer operation. If both source and destination are registers, then the CPU simply causes data to be transferred from one register to another; this is an operation internal to the CPU.

If one or both operands are in memory, then the CPU must perform some or all of the following actions:

- a) Calculate the memory address, based on the addressing mode.
- b) If the address refers to virtual memory, translate from virtual to actual memory address.
- c) Determine whether the addressed item is in cache.
- d) If not, issue a command to the memory module.

yin ny

Commonly used data transfer operation:

Operation	Name Description
Move (Transfer)	Transfer word or block from <u>source to destination</u> .
Store	Transfer word from <u>processor to memory</u> .
Load (fetch)	Transfer word from <u>memory to processor</u> .
Exchange	Swap contents of <u>source and destination</u> .
Clear (reset)	Transfer <u>word of 0s to destination</u> .
Set	Transfer <u>word of 1s to destination</u> .
Push	Transfer word from <u>source to top of stack</u> .
Pop	Transfer word from <u>top of stack to destination</u> .

3.7.2 Arithmetic:

Most machines provide the basic arithmetic operations like add, subtract, multiply, divide etc. These are invariably provided for signed integer (fixed-point) numbers. They are also available for floating point number.

The execution of an arithmetic operation may involve data transfer operation to provide the operands to the ALU input and to deliver the result of the ALU operation.

Commonly used data transfer operation:

Operation	Name Description
<u>Add</u>	Compute sum of two operands
<u>Subtract</u>	Compute difference of two operands
<u>Multiply</u>	Compute product of two operands
<u>Divide</u>	Compute quotient of two operands
<u>Absolute</u>	Replace operand by its absolute value
<u>Negate</u>	Change sign of operand
<u>Increment</u>	Add 1 to operand
<u>Decrement</u>	Subtract 1 from operand

3.7.3 Logical:

Most machines also provide a variety of operations for manipulating individual bits of a word or other addressable units often referred to as "bit twiddling". They are based upon Boolean operations

Most commonly available logical operations are:

Operation	Name Description
<u>AND</u>	Performs the logical operation AND bitwise

D 94651

OR	→ Performs the logical operation OR bitwise
NOT	→ Performs the logical operation NOT bitwise
Exclusive OR	→ Performs the specified logical operation Exclusive-OR bitwise
Test	→ Test specified condition; set flag(s) based on outcome
Compare	→ Make logical or arithmetic comparison Set flag(s) based on outcome
Shift Left (right)	→ Shift operand, introducing constant at end
Rotate Left (right)	→ Shift operation, with wraparound end

3.7.4 Conversion:

Conversion instructions are those that change the format or operate on the format of data. An example is converting from decimal to binary.

3.7.5 Input / Output:

Input / Output instructions are used to transfer data between input/output devices and memory/CPU register.

Commonly available I/O operations are:

Operation	Name Description
Input (Read)	→ Transfer data from specified I/O port or device to destination (e.g., main memory or processor register)
Output (Write)	→ Transfer data from specified source to I/O port or device.
Start I/O	→ Transfer instructions to I/O processor to initiate I/O operation.
Test I/O	→ Transfer status information from I/O system to specified destination

3.7.6 System Control:

System control instructions are those which are used for system setting and it can be used only in privileged state. Typically, these instructions are reserved for the use of operating systems. For example, a system control instruction may read or alter the content of a control register. Another instruction may be to read or modify a storage protection key.

3.7.7 Transfer of Control:

In most of the cases, the next instruction to be performed is the one that immediately follows the current instruction in memory. Therefore, program counter helps us to get the next instruction. But sometimes it is required to change the sequence of instruction execution and for that instruction set should provide instructions to accomplish these tasks. For these instructions, the operation performed by the CPU is to upload the program counter to contain the address of some instruction in memory. The most common transfer-of-control operations found in instruction set are: branch, skip and procedure call.

3.7.7.1 Branch Instruction

Conditional It is made only if a certain condition is met

unconditional It is made by updating the Program counter to address specified in operand

A branch instruction, also called a jump instruction, has one of its operands as the address of the next instruction to be executed. Basically there are two types of branch instructions: Conditional Branch instruction and unconditional branch instruction. In case of unconditional branch instruction, the branch is made by updating the program counter to address specified in operand. In case of conditional branch instruction, the branch is made only if a certain condition is met. Otherwise, the next instruction in sequence is executed.

There are two common ways of generating the condition to be tested in a conditional branch instruction

First most machines provide a 1-bit or multiple-bit condition code that is set as the result of some operations. As an example, an arithmetic operation could set a 2-bit condition code with one of the following four values: zero, positive, negative and overflow. On such a machine, there could be four different conditional branch instructions:

BRP X Branch to location X if result is positive

BRN X Branch to location X if result is negative

BRZ X Branch to location X if result is zero

BRO X Branch to location X if overflow occurs

In all of these cases, the result referred to is the result of the most recent operation that set the condition code.

Another approach that can be used with three address instruction format is to perform a comparison and specify a branch in the same instruction.

For example,

BRE R1, R2, X Branch to X if contents of R1 = Contents of R2.

3.7.7.2 Skip Instruction

Another common form of transfer-of-control instruction is the skip instruction. Generally, the skip implies that one instruction to be skipped; thus the implied address equals the address of the next instruction plus one instruction length. A typical example is the increment-and-skip-if-zero (ISZ) instruction. For example,

ISZ R1

This instruction will increment the value of the register R1. If the result of the increment is zero, then it will skip the next instruction.

3.7.7.3 Procedure Call Instruction

IL - Instruction length
 RN -
 PC - Program counter

A procedure is a self-contained computer program that is incorporated into a large program. At any point in the program the procedure may be invoked, or called. The processor is instructed to go and execute the entire procedure and then return to the point from which the call took place. The procedure mechanism involves two basic instructions: a call instruction that branches from the present location to the procedure, and a return instruction that returns from the procedure to the place from which it was called. Both of these are forms of branching instructions.

Some important points regarding procedure call:

- A procedure can be called from more than one location.
- A procedure call can appear in a procedure. This allows the nesting of procedures to an arbitrary depth.
- Each procedure call is matched by a return in the called program.

Since we can call a procedure from a variety of points, the CPU must somehow save the return address so that the return can take place appropriately. There are three common places for storing the return address:

- Register
 - Start of procedure
 - Top of stack
- } common places for storing the return address.

Consider a machine language instruction CALL X, which stands for call procedure at location X. If the register approach is used, CALL X causes the following actions:

$RN \leftarrow PC + IL$

$PC \leftarrow X$

where RN is a register that is always used for this purpose, PC is the program counter and IL is the instruction length.

The called procedure can now save the contents of RN to be used for the later return.

A second possibility is to store the return address at the start of the procedure. In this case, CALL X causes

$X \leftarrow PC + IL$

$PC \leftarrow X + 1$

Both of these approaches have been used. The only limitation of these approaches is that they prevent the use of reentrant procedures. A reentrant procedure is one in which it is possible to have several calls open to it at the same time.

A more general approach is to use stack. When the CPU executes a call, it places the return address on the stack. When it executes a return, it uses the address on the stack.

Stack frame is the entire set of parameters, including return address that is stored for a procedure invocation.

It may happen that, the called procedure might have to use the processor registers. This will overwrite the contents of the registers and the calling environment will lose the information. So, it is necessary to preserve the contents of processor register too along with the return address. The stack is used to store the contents of processor register. On return from the procedure call, the contents of the stack will be popped out to appropriate registers.

In addition to provide a return address, it is also often necessary to pass parameters with a procedure call. The most general approach to parameter passing is the stack. When the processor executes a call, it not only stacks the return address, it stacks parameters to be passed to the called procedures. The called procedure can access the parameters from the stack. Upon return, return parameters can also be placed on the stack. The entire set of parameters, including return address that is stored for a procedure invocation is referred to as stack frame.

Most commonly used transfer of control operation:

Operation	Name Description
Jump (branch)	Unconditional transfer, load PC with specific address
Jump conditional	Test specific condition; either load PC with specific address or do nothing, based on condition
Jump to subroutine	Place current program control information in known location; jump to specific address
Return	Replace contents of PC and other register from known location
Skip	Increment PC to skip next instruction
Skip Conditional	Test specified condition; either skip or do nothing based on condition
Halt	Stop program execution

3.8 Instruction Format:

An instruction format defines the layout of the bits of an instruction, in terms of its constituents parts. An instruction format must include an opcode and, implicitly or explicitly, zero or more operands. Each explicit operand is referenced using one of the addressing mode that is available for that machine. The format must, implicitly or explicitly, indicate the addressing mode of each operand. For most instruction sets, more than one instruction format is used. Four common instruction format are shown in the figure on the next slide .

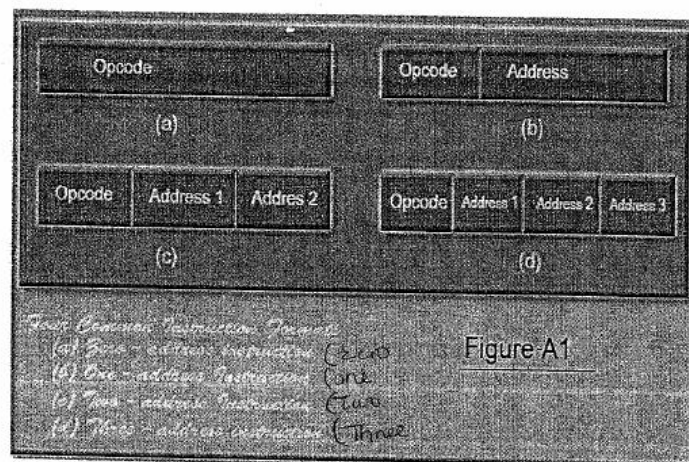


Figure - A1

3.9 Instruction Length:

On some machines, all instructions have the same length; on others there may be many different lengths. Instructions may be shorter than, the same length as, or more than the word length. Having all the instructions be the same length is simpler and make decoding easier but often wastes space, since all instructions then have to be as long as the longest one. Possible relationship between instruction length and word length is shown in the figure.

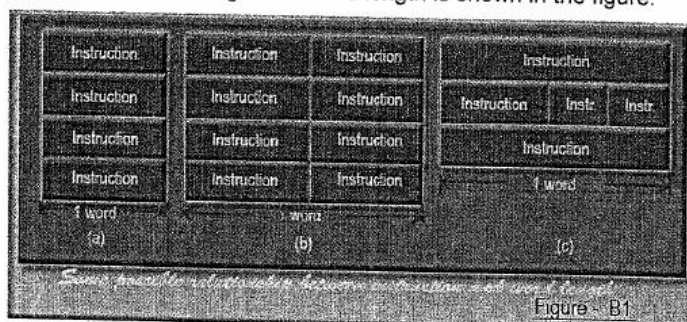


Figure - B1

Generally there is a correlation between memory transfer length and the instruction length. Either the instruction length should be equal to the memory transfer length or one should be a multiple of the other. Also in most of the case there is a correlation between memory transfer length and word length of the machine.

3.10 Allocation of Bits:

For a given instruction length, there is a clearly a trade-off between the number of opcodes and the power of the addressing capabilities. More opcodes obviously mean more bits in the opcode

field. For an instruction format of a given length, this reduces the number of bits available for addressing.

The following interrelated factors go into determining the use of the addressing bits:

Number of Addressing modes:

Sometimes an addressing mode can be indicated implicitly. In other cases, the addressing mode must be explicit, and one or more bits will be needed.

Number of Operands:

Typical instructions on today's machines provide for two operands. Each operand address in the instruction might require its own mode indicator, or the use of a mode indicator could be limited to just one of the address fields.

Register versus memory:

A machine must have registers so that data can be brought into the CPU for processing. With a single user-visible register (usually called the accumulator), one operand address is implicit and consumes no instruction bits. Even with multiple registers, only a few bits are needed to specify the register. The more registers that can be used for operand references, the fewer bits are needed.

Number of register sets:

A number of machines have one set of general purpose registers, with typically 8 or 16 registers in the set. These registers can be used to store data and can be used to store addresses for displacement addressing. The trend recently has been away from one bank of general purpose registers and toward a collection of two or more specialized sets (such as data and displacement).

Address range:

For addresses that reference memory, the range of addresses that can be referenced is related to the number of address bits. With displacement addressing, the range is opened up to the length of the address register.

Address granularity:

In a system with 16- or 32-bit words, an address can reference a word or a byte at the designer's choice. Byte addressing is convenient for character manipulation but requires, for a fixed size memory, more address bits.

Variable-Length Instructions:

Instead of looking for fixed length instruction format, designer may choose to provide a variety of instructions formats of different lengths. This tactic makes it easy to provide a large repertoire of opcodes, with different opcode lengths.

Addressing can be more flexible, with various combinations of register and memory references plus addressing modes. With variable length instructions, many variations can be provided efficiently and compactly. The principal price to pay for variable length instructions is an increase in the complexity of the CPU.

Number of addresses:

The processor architecture is described in terms of the number of addresses contained in each instruction. Most of the arithmetic and logic instructions will require more operands. All arithmetic and logic operations are either unary (one source operand, e.g. NOT) or binary (two source operands, e.g. ADD).

Thus, we need a maximum of two addresses to reference source operands. The result of an operation must be stored, suggesting a third reference. Three address instruction formats are not common because they require a relatively long instruction format to hold the three address reference.

With two address instructions, and for binary operations, one address must do double duty as both an operand and a result.

In one address instruction format, a second address must be implicit for a binary operation. For implicit reference, a processor register is used and it is termed as accumulator (AC). The accumulator contains one of the operands and is used to store the result.

Consider a simple arithmetic expression to evaluate:

$$Y = (A + B) / (C * D)$$

Instruction	Comment
ADD Y, A, B	Y ← A + B
MULT Z, C, D	Z ← C * D
DIV Y, Y, Z	Y ← Y / Z

Instruction	Comment
MOV Y, A	Y ← A
ADD Y, B	Y ← Y + B
MOV Z, C	Z ← C
MULT Z, D	Z ← Z * D
DIV Y, Z	Y ← Y / Z

Instruction	Comment
LOAD C	AC ← C
MULT D	AC ← AC * D
STORE Y	Y ← AC
LOAD A	AC ← A
ADD B	AC ← AC + B
DIV Y	AC ← AC / Y
STORE Y	Y ← AC

3.11 Instruction Addressing

We have examined the types of *operands* and *operations* that may be specified by *machine instructions*. Now we have to see how is the address of an operand specified, and how are the *bits* of an instruction organized to define the *operand addresses* and operation of that instruction.

3.12 Addressing Modes:

The most common addressing techniques are:

- Immediate
- Direct
- Indirect
- Register
- Register Indirect
- Displacement
- Stack

All computer architectures provide more than one of these *addressing modes*. The question arises as to how the control unit can determine which addressing mode is being used in a particular instruction. Several approaches are used. Often, different *op-codes* will use different addressing modes. Also, one or more bits in the *instruction format* can be used as a mode field. The value of the mode field determines which addressing mode is to be used.

What is the interpretation of *effective address (EA)*. In a system without virtual memory, the effective address will be either a main memory address or a register. In a virtual memory system, the effective address is a virtual address or a register. The actual mapping to a physical address is a function of the paging mechanism and is invisible to the programmer.

To explain the addressing modes, we use the following notation:

A = contents of an address field in the instruction that refers to a memory

R = contents of an address field in the instruction that refers to a register

EA = actual (effective) address of the location containing the referenced operand

(X) = contents of location X

3.12.1 Immediate Addressing:

The simplest form of addressing is immediate addressing, in which the operand is actually present in the instruction:

$$\text{OPERAND} = A$$

This mode can be used to define and use constants or set initial values of variables. The advantage of immediate addressing is that no memory reference other than the instruction fetch is required to obtain the operand. The disadvantage is that the size of the number is restricted to the size of the address field, which, in most instruction sets, is small compared with the word length.

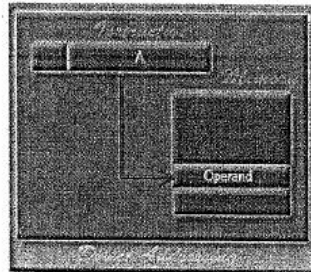


3.12.2 Direct Addressing:

A very simple form of addressing is direct addressing, in which the address field contains the effective address of the operand:

$$EA = A$$

It requires only one memory reference and no special calculation. The obvious limitation is that it provides only a limited address space.



3.12.3 Indirect Addressing:

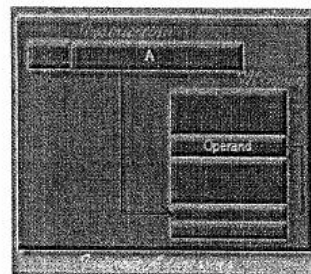
With direct addressing, the length of the address field is usually less than the word length, thus limiting the address range. One solution is to have the address field refer to the address of a word in memory, which in turn contains a full-length address of the operand. This is known as indirect addressing:

$$EA = (A)$$

The parentheses are to be interpreted as meaning "contents of". The obvious advantage of this approach is that for a word length of N , an address space of 2^N is now available. The disadvantage is that instruction execution requires two memory references to fetch the operand; one to get its address and the other to get its value.

effective address

Although, the number that can be addressed, now equal to 2^N , the number of different EAs that may be referenced at any one time is limited to 2^K , where K is the length of the address field.

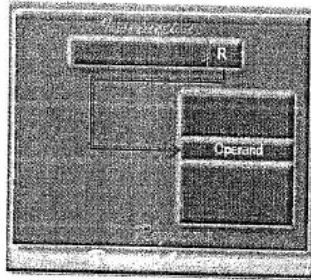


3.12.4 Register Addressing:

Register addressing is similar to direct addressing. The only difference is that the address field refers to a register rather than a main memory address:

$$EA = R$$

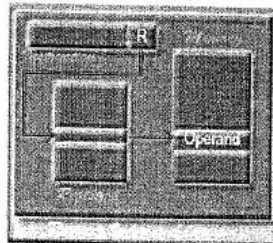
The advantages of register addressing are that only a small address field is needed in the instruction and no memory reference is required. The disadvantage of register addressing is that the address space is very limited.

**3.12.5 Register Indirect Addressing:**

Register indirect addressing is similar to indirect addressing, except that the address field refers to a register instead of a memory location. It requires only one memory reference and no special calculation.

$$EA = (R)$$

Register indirect addressing uses one less memory reference than indirect addressing. Because, the first information is available in a register which is nothing but a memory address. From that memory location, we use to get the data or information. In general, register access is much more faster than the memory access.

**3.12.6 Displacement Addressing:**

A very powerful mode of addressing combines the capabilities of direct addressing and register indirect addressing, which is broadly categorized as displacement addressing:

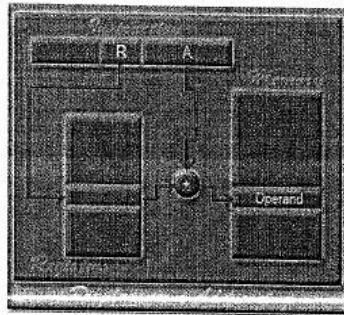
$$EA = A + (R)$$

Displacement (pointing to A)
Register value (pointing to R)

Displacement addressing requires that the instruction have two address fields, at least one of which is explicit. The value contained in one address field (value = A) is used directly. The other address field, or an implicit reference based on op-code, refers to a register whose contents are added to A to produce the effective address.

Three of the most common use of displacement addressing are:

- Relative addressing
- Base-register addressing
- Indexing



3.12.6.1 Relative Addressing:

For relative addressing, the implicitly referenced register is the program counter (PC). That is, the current instruction address is added to the address field to produce the EA. Thus, the effective address is a displacement relative to the address of the instruction.

3.12.6.2 Base-Register Addressing:

The reference register contains a memory address, and the address field contains a displacement from that address. The register reference may be *explicit* or *implicit*.

In some implementation, a single segment/base register is employed and is used implicitly. In others, the programmer may choose a register to hold the base address of a segment, and the instruction must reference it explicitly.

3.12.6.3 Indexing:

The address field references a main memory address, and the reference register contains a positive displacement from that address. In this case also the register reference is sometimes explicit and sometimes implicit.

Generally index register are used for iterative tasks, it is typical that there is a need to increment or decrement the index register after each reference to it. Because this is such a common operation, some system will automatically do this as part of the same instruction cycle.

This is known as *auto-indexing*. We may get two types of auto-indexing:

- one is auto-incrementing and the other one is
- auto-decrementing.

If certain registers are devoted exclusively to indexing, then auto-indexing can be invoked implicitly and automatically. If general purpose register are used, the auto-index operation may need to be signaled by a bit in the instruction.

Auto-indexing using *increment* can be depicted as follows:

$$\begin{aligned} EA &= A + (R) \\ R &= (R) + 1 \end{aligned}$$

Auto-indexing using *decrement* can be depicted as follows:

$$\begin{aligned} EA &= A + (R) \\ R &= (R) - 1 \end{aligned}$$

In some machines, both *indirect addressing* and *indexing* are provided, and it is possible to employ both in the same instruction. There are two possibilities: The indexing is performed either before or after the indirection. If indexing is performed after the indirection, it is termed *post-indexing*

$$EA = (A) + (R)$$

First, the contents of the address field are used to access a memory location containing an address. This address is then indexed by the register value. With pre-indexing, the indexing is performed before the indirection:

$$EA = (A + (R))$$

An address is calculated, the calculated address contains not the operand, but the address of the operand.

3.13 Stack Addressing:

A stack is a linear array or list of locations. It is sometimes referred to as a *pushdown list* or *first-in-last-out* last-in-first-out queue. A stack is a reserved block of locations. Items are appended to the top of the stack so that, at any given time, the block is partially filled. Associated with the stack is a

pointer whose value is the address of the top of the stack. The stack pointer is maintained in a register. Thus, references to stack locations in memory are in fact register indirect addresses. The stack mode of addressing is a form of implied addressing. The machine instructions need not include a memory reference but implicitly operate on the top of the stack.

CHAPTER FOUR

MEMORY AND MEMORY MANAGEMENT

4.1 Concept of Memory

We have already mentioned that digital computer works on stored programmed concept introduced by Von Neumann. We use memory to store the information, which includes both program and data. Due to several reasons, we have different kind of memories. We use different kind of memory at different level.

The memory of computer is broadly categories into two categories:

- Internal and
- external

Internal memory is used by CPU to perform task and external memory is used to store bulk information, which includes large software and data. Memory is used to store the information in digital form. The memory hierarchy is given by:

- Register
- Cache Memory
- Main Memory
- Magnetic Disk
- Removable media (Magnetic tape)

Register:

This is a part of Central Processor Unit, so they reside inside the CPU. The information from main memory is brought to CPU and keep the information in register. Due to space and cost constraints, we have got a limited number of registers in a CPU. These are basically faster devices.

Cache Memory:

Cache memory is a storage device placed in between CPU and main memory. These are semiconductor memories. These are basically fast memory device, faster than main memory. We cannot have a big volume of cache memory due to its higher cost and some constraints of the CPU. Due to higher cost we cannot replace the whole main memory by faster memory. Generally, the most recently used information is kept in the cache memory. It is brought from the main memory and placed in the cache memory. Now a days, we get CPU with internal cache.

Main Memory:

Like cache memory, main memory is also semiconductor memory. But the main memory is relatively slower memory. We have to first bring the information (whether it is data or program), to main memory. CPU can work with the information available in main memory only.

Magnetic Disk:

This is bulk storage device. We have to deal with huge amount of data in many application. But we don't have so much semiconductor memory to keep these information in our computer. On the other hand, semiconductor memories are volatile in nature. It loses its content once we switch off the computer. For permanent storage, we use magnetic disk. The storage capacity of magnetic disk is very high.

Removable media:

For different application, we use different data. It may not be possible to keep all the information in magnetic disk. So, whichever data we are not using currently, can be kept in removable media. Magnetic tape is one kind of removable medium. CD is also a removable media, which is an optical device.

Register, cache memory and main memory are internal memory. Magnetic Disk, removable media are external memory. Internal memories are semiconductor memory. Semiconductor memories are categorized as volatile memory and non-volatile memory.

RAM: Random Access Memories are volatile in nature. As soon as the computer is switched off, the contents of memory are also lost.

ROM: Read only memories are non-volatile in nature. The storage is permanent, but it is read only memory. We cannot store new information in ROM.

Several types of ROM are available:

- **PROM:** Programmable Read Only Memory; it can be programmed once as per user requirements.
- **EPROM:** Erasable Programmable Read Only Memory; the contents of the memory can be erased and store new data into the memory. In this case, we have to erase whole information.
- **EEPROM:** Electrically Erasable Programmable Read Only Memory; in this type of memory the contents of a particular location can be changed without effecting the contents of other location.

4.2 Main Memory

The main memory of a computer is semiconductor memory. The main memory unit of computer is basically consists of two kinds of memory:

RAM: Random access memory; which is volatile in nature.

ROM: Read only memory; which is non-volatile.

The permanent information are kept in ROM and the user space is basically in RAM. The smallest unit of information is known as bit (binary digit), and in one memory cell we can store one bit of information. 8 bit together is termed as a byte.

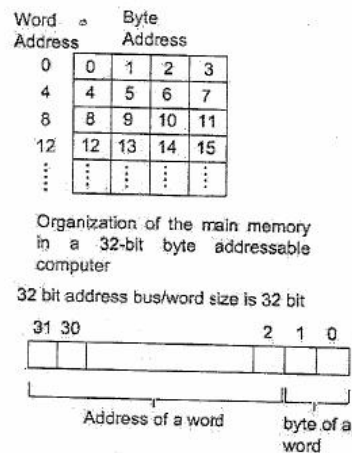
The maximum size of main memory that can be used in any computer is determined by the addressing scheme. A computer that generates 16-bit address is capable of addressing up to 2^{16} which is equal to 64K memory location. Similarly, for 32 bit addresses, the total capacity will be 2^{32} which is equal to 4G memory location.

In some computer, the smallest addressable unit of information is a memory word and the machine is called word addressable.

In some computer, individual address is assigned for each byte of information, and it is called byte-addressable computer. In this computer, one memory word contains one or more memory bytes which can be addressed individually. A byte addressable 32-bit computer, each memory word contains 4 bytes. A possible way of address assignment is shown in figure below. The address of a word is always integer multiple of 4.

The main memory is usually designed to store and retrieve data in word length quantities. The word length of a computer is generally defined by the number of bits actually stored or retrieved in one main memory access.

Consider a machine with 32 bit address bus. If the word size is 32 bit, then the high order 30 bit will specify the address of a word. If we want to access any byte of the word, then it can be specified by the lower two bit of the address bus.



The data transfer between main memory and the CPU takes place through two CPU registers.

- MAR : Memory Address Register
- MDR : Memory Data Register.

If the MAR is k-bit long, then the total addressable memory location will be 2^k . If the MDR is n-bit long, then the n bit of data is transferred in one memory cycle. The transfer of data takes place through memory bus, which consist of address bus and data bus. In the above example, size of data bus is n-bit and size of address bus is k bit. It also includes control lines like Read, Write and Memory Function Complete (MFC) for coordinating data transfer. In the case of byte addressable computer, another control line to be added to indicate the byte transfer instead of the whole word.

For memory operation, the CPU initiates a memory operation by loading the appropriate data i.e., address to MAR. If it is a memory read operation, then it sets the read memory control line to 1. Then the contents of the memory location is brought to MDR and the memory control circuitry indicates this to the CPU by setting MFC to 1.

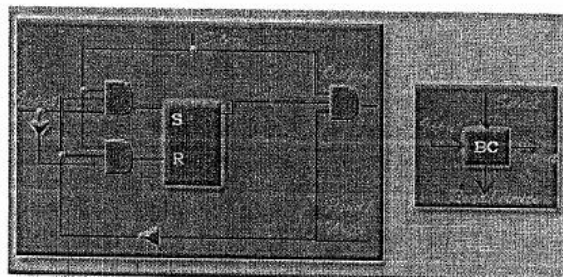
If the operation is a memory write operation, then the CPU places the data into MDR and sets the write memory control line to 1. Once the contents of MDR are stored in specified memory location, then the memory control circuitry indicates the end of operation by setting MFC to 1.

A useful measure of the speed of memory unit is the time that elapses between the initiation of an operation and the completion of the operation (for example, the time between Read and MFC). This is referred to as **Memory Access Time**. Another measure is memory cycle time. This is the minimum time delay between the initiation two independent memory operations (for

example, two successive memory read operation). Memory cycle time is slightly larger than memory access time.

4.3 Binary Storage Cell:

The binary storage cell is the basic building block of a memory unit. The binary storage cell that stores one bit of information can be modelled by an SR latch with associated gates. This model of binary storage cell is shown in the figure.



4.3.1 1 bit Binary Cell (BC)

The binary cell stores one bit of information in its internal latch. Control input to binary cell

Select	Read/Write	Memory Operation
0	X	None
1	0	Write
1	1	Read

The storage part is modelled here with SR-latch, but in reality it is an electronics circuit made up of transistors. The memory constructed with the help of transistors is known as semiconductor memory. The semiconductor memories are termed as Random Access Memory (RAM), because it is possible to access any memory location in random.

Depending on the technology used to construct a RAM, there are two types of RAM -

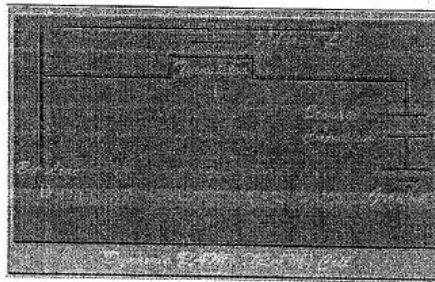
SRAM: Static Random Access Memory.

DRAM: Dynamic Random Access Memory.

4.3.2 Dynamic Ram (DRAM):

A DRAM is made with cells that store data as charge on capacitors. The presence or absence of charge in a capacitor is interpreted as binary 1 or 0. Because capacitors have a natural tendency to discharge due to leakage current, dynamic RAM require periodic charge refreshing to maintain data storage. The term dynamic refers to this tendency of the stored charge to leak away, even with power continuously applied.

A typical DRAM structure for an individual cell that stores one bit information is shown in the figure.



For the write operation, a voltage signal is applied to the bit line B, a high voltage represents 1 and a low voltage represents 0. A signal is then applied to the address line, which will turn on the transistor T, allowing a charge to be transferred to the capacitor.

For the read operation, when a signal is applied to the address line, the transistor T turns on and the charge stored on the capacitor is fed out onto the bit line B.

4.3.3 Static RAM (SRAM):

In an SRAM, binary values are stored using traditional flip-flop constructed with the help of transistors. A static RAM will hold its data as long as power is supplied to it.

A typical SRAM constructed with transistors is shown in the figure.

Four transistors (T₁, T₂, T₃, T₄) are cross connected in an arrangement that produces a stable logic state. In logic state 1, point A₁ is high and point A₂ is low; in this state T₁ and T₄ are off, and T₂ and T₃ are on. In logic state 0, point A₁ is low and point A₂ is high; in this state T₁ and T₄ are on, and T₂ and T₃ are off.

Both states are stable as long as the dc supply voltage is applied. The address line is used to open or close a switch which is nothing but another transistor. The address line controls two transistors (T₅ and T₆). When a signal is applied to this line, the two transistors are switched on, allowing a read or write operation.

For a write operation, the desired bit value is applied to line B, and its complement is applied to line $\bar{B}$. This forces the four transistors (T1, T2, T3, T4) into the proper state.

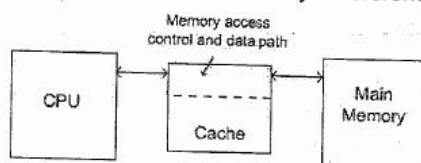
For a read operation, the bit value is read from the line B. When a signal is applied to the address line, the signal of point A1 is available in the bit line B.

4.3.4 SRAM Versus DRAM :

- Both static and dynamic RAMs are volatile, that is, it will retain the information as long as power supply is applied.
- A dynamic memory cell is simpler and smaller than a static memory cell. Thus a DRAM is more dense, i.e., packing density is high (more cell per unit area). DRAM is less expensive than corresponding SRAM.
- DRAM requires the supporting refresh circuitry. For larger memories, the fixed cost of the refresh circuitry is more than compensated for by the less cost of DRAM cells
- SRAM cells are generally faster than the DRAM cells. Therefore, to construct faster memory modules (like cache memory) SRAM is used.

4.4 Cache Memory

Analysis of large number of programs has shown that a number of instructions are executed repeatedly. This may be in the form of a simple loops, nested loops, or a few procedures that repeatedly call each other. It is observed that many instructions in each of a few localized areas of the program are repeatedly executed, while the remainder of the program is accessed relatively less. This phenomenon is referred to as locality of reference.



Cache memory between the CPU and the main memory

Now, if it can be arranged to have the active segments of a program in a fast memory, then the total execution time can be significantly reduced. It is the fact that CPU is a faster device and memory is a relatively slower device. Memory access is the main bottleneck for the performance efficiency. If a faster memory device can be inserted between main memory and CPU, the efficiency can be increased. The faster memory that is inserted between CPU and Main Memory is termed as Cache memory. To make this arrangement effective, the cache must be considerably faster than the main memory, and typically it is 5 to 10 time faster than the main

memory. This approach is more economical than the use of fast memory device to implement the entire main memory. This is also a feasible due to the locality of reference that is present in most of the program, which reduces the frequent data transfer between main memory and cache memory.

4.4.1 Operation of Cache Memory

The memory control circuitry is designed to take advantage of the property of locality of reference. Some assumptions are made while designing the memory control circuitry:

1. The CPU does not need to know explicitly about the existence of the cache.
2. The CPU simply makes Read and Write request. The nature of these two operations are same whether cache is present or not.
3. The address generated by the CPU always refer to location of main memory.
4. The memory access control circuitry determines whether or not the requested word currently exists in the cache.

When a Read request is received from the CPU, the contents of a block of memory words containing the location specified are transferred into the cache. When any of the locations in this block is referenced by the program, its contents are read directly from the cache.

The cache memory can store a number of such blocks at any given time. The correspondence between the Main Memory Blocks and those in the cache is specified by means of a mapping function. When the cache is full and a memory word is referenced that is not in the cache, a decision must be made as to which block should be removed from the cache to create space to bring the new block to the cache that contains the referenced word. Replacement algorithms are used to make the proper selection of block that must be replaced by the new one. When a write request is received from the CPU, there are two ways that the system can proceed. In the first case, the cache location and the main memory location are updated simultaneously. This is called the store through method or write through method.

The alternative is to update the cache location only. During replacement time, the cache block will be written back to the main memory. If there is no new write operation in the cache block, it is not required to write back the cache block in the main memory. This information can be kept with the help of an associated bit. This bit is set while there is a write operation in the cache block. During replacement, it checks this bit, if it is set, then write back the cache block in main memory otherwise not. This bit is known as dirty bit. If the bit gets dirty (set to one), writing to main memory is required.

The alternative is to update the cache location only.

This write through method is simpler, but it results in unnecessary write operations in the main memory when a given cache word is updated a number of times during its cache residency period.

Consider the case where the addressed word is not in the cache and the operation is a read. First the block of the words is brought to the cache and then the requested word is forwarded to the CPU. But it can be forwarded to the CPU as soon as it is available to the cache, instead of whole block to be loaded into the cache. This is called load through, and there is some scope to save time while using load through policy. During a write operation, if the address word is not in the cache, the information is written directly into the main memory. A write operation normally refers to the location of data areas and the property of locality of reference is not as pronounced in accessing data when write operation is involved. Therefore, it is not advantageous to bring the data block to the cache when there a write operation, and the addressed word is not present in cache.

4.5 Mapping Functions

The mapping functions are used to map a particular block of main memory to a particular block of cache. This mapping function is used to transfer the block from main memory to cache memory. Three different mapping functions are available:

4.5.1 Direct mapping:

A particular block of main memory can be brought to a particular block of cache memory. So, it is not flexible.

4.5.2 Associative mapping:

In this mapping function, any block of Main memory can potentially reside in any cache block position. This is much more flexible mapping method.

4.5.3 Block-set-associative mapping:

In this method, blocks of cache are grouped into sets, and the mapping allows a block of main memory to reside in any block of a specific set. From the flexibility point of view, it is in between to the other two methods.

All these three mapping methods are explained with the help of an example. Consider a cache of 4096 (4K) words with a block size of 32 words. Therefore, the cache is organized as 128 blocks. For 4K words, required address lines are 12 bits. To select one of the block out of 128 blocks, we need 7 bits of address lines and to select one word out of 32 words, we need 5 bits of address lines. So the total 12 bits of address is divided for two groups, lower 5 bits are used to select a word within a block, and higher 7 bits of address are used to select any block of cache memory.

Let us consider a main memory system consisting 64K words. The size of address bus is 16 bits. Since the block size of cache is 32 words, so the main memory is also organized as block size of 32 words. Therefore, the total number of blocks in main memory is 2048 ($2K \times 32 \text{ words} = 64K \text{ words}$). To identify any one block of 2K blocks, we need 11 address lines. Out of 16 address lines of main memory, lower 5 bits are used to select a word within a block and higher 11 bits are used to select a block out of 2048 blocks. Number of blocks in cache memory is 128 and number of blocks in main memory is 2048, so at any instant of time only 128 blocks out of 2048 blocks can reside in cache memory. Therefore, we need mapping function to put a particular block of main memory into appropriate block of cache memory.

4.5.4 Direct Mapping Technique:

The simplest way of associating main memory blocks with cache block is the direct mapping technique. In this technique, block k of main memory maps into block $k \text{ modulo } m$ of the cache, where m is the total number of blocks in cache. In this example, the value of m is 128. In direct mapping technique, one particular block of main memory can be transferred to a particular block of cache which is derived by the modulo function.

Since more than one main memory block is mapped onto a given cache block position, contention may arise for that position. This situation may occur even when the cache is not full. Contention is resolved by allowing the new block to overwrite the currently resident block. So the replacement algorithm is trivial.

The detail operation of direct mapping technique is as follows:

* The main memory address is divided into three fields. The field size depends on the memory capacity and the block size of cache. In this example, the lower 5 bits of address is used to identify a word within a block. Next 7 bits are used to select a block out of 128 blocks (which is the capacity of the cache). The remaining 4 bits are used as a TAG to identify the proper block of main memory that is mapped to cache.

When a new block is first brought into the cache, the high order 4 bits of the main memory address are stored in four TAG bits associated with its location in the cache. When the CPU generates a memory request, the 7-bit block address determines the corresponding cache block. The TAG field of that block is compared to the TAG field of the address. If they match, the desired word specified by the low-order 5 bits of the address is in that block of the cache. If there is no match, the required word must be accessed from the main memory, that is, the contents of that block of the cache is replaced by the new block that is specified by the new address generated by the CPU and correspondingly the TAG bit will also be changed by the

high order 4 bits of the address. The whole arrangement for direct mapping technique is shown in the figure.

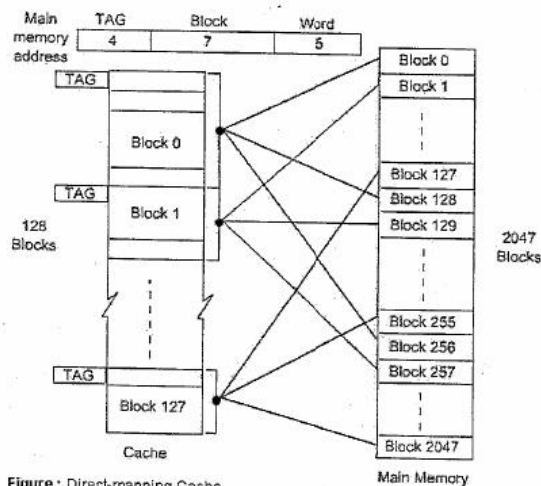
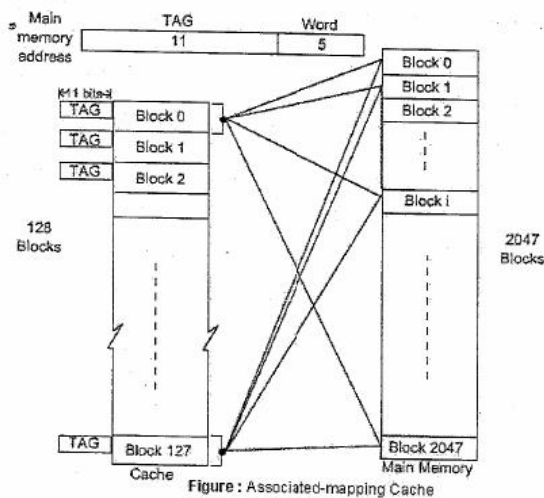


Figure: Direct-mapping Cache

4.5.5 Associated Mapping Technique:

In the associative mapping technique, a main memory block can potentially reside in any cache block position. In this case, the main memory address is divided into two groups, low-order bits identifies the location of a word within a block and high-order bits identifies the block. In the example here, 11 bits are required to identify a main memory block when it is resident in the cache, high-order 11 bits are used as TAG bits and low-order 5 bits are used to identify a word within a block. The TAG bits of an address received from the CPU must be compared to the TAG bits of each block of the cache to see if the desired block is present.

In the associative mapping, any block of main memory can go to any block of cache, so it has got the complete flexibility and we have to use proper replacement policy to replace a block from cache if the currently accessed block of main memory is not present in cache. It might not be practical to use this complete flexibility of associative mapping technique due to searching overhead, because the TAG field of main memory address has to be compared with the TAG field of all the cache block. In this example, there are 128 blocks in cache and the size of TAG is 11 bits. The whole arrangement of Associative Mapping Technique is shown in the figure.



4.5.6 Block-Set-Associative Mapping Technique:

This mapping technique is intermediate to the above two techniques. Blocks of the cache are grouped into sets, and the mapping allows a block of main memory to reside in any block of a specific set. Therefore, the flexibility of associative mapping is reduced from full freedom to a set of specific blocks. This also reduces the searching overhead, because the search is restricted to number of sets, instead of number of blocks. Also the contention problem of the direct mapping is eased by having a few choices for block replacement.

Consider the same cache memory and main memory organization of the previous example. Organize the cache with 4 blocks in each set. The TAG field of associative mapping technique is divided into two groups, one is termed as SET bit and the second one is termed as TAG bit. Since each set contains 4 blocks, total number of set is 32. The main memory address is grouped into three parts: low-order 5 bits are used to identifies a word within a block. Since there are total 32 sets present, next 5 bits are used to identify the set. High-order 6 bits are used as TAG bits. The 5-bit set field of the address determines which set of the cache might contain the desired block. This is similar to direct mapping technique, in case of direct mapping, it looks for block, but in case of block-set-associative mapping, it looks for set. The TAG field of the address must then be compared with the TAGs of the four blocks of that set. If a match occurs, then the block is present in the cache; otherwise the block containing the addressed word must be brought to the cache. This block will potentially come to the corresponding set only. Since,

57

there are four blocks in the set, we have to choose appropriately which block to be replaced if all the blocks are occupied. Since the search is restricted to four blocks only, so the searching complexity is reduced. The whole arrangement of block-set-associative mapping technique is shown in the figure. It is clear that if we increase the number of blocks per set, then the number of bits in SET field is reduced. Due to the increase of blocks per set, complexity of search is also increased. The extreme condition of 128 blocks per set requires no set bits and corresponds to the fully associative mapping technique with 11 TAG bits. The other extreme of one block per set is the direct mapping method.

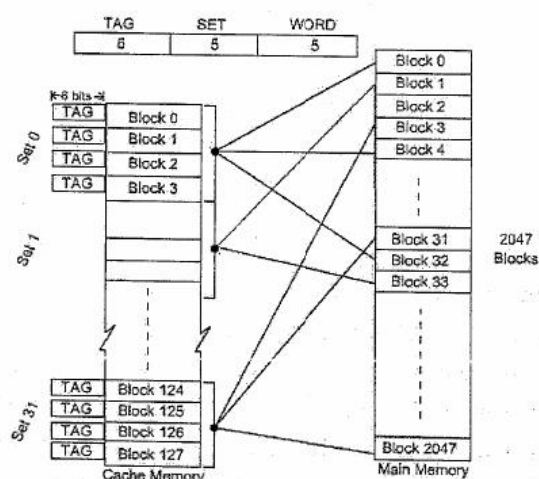


Figure : Block-set-Associative mapping Cache with 4 blocks per set

4.6 Replacement Algorithms (Replacement)

When a new block must be brought into the cache and all the positions that it may occupy are full, a decision must be made as to which of the old blocks is to be overwritten. In general, a policy is required to keep the block in cache when they are likely to be referenced in near future. However, it is not easy to determine directly which of the block in the cache are about to be referenced. The property of locality of reference gives some clue to design good replacement policy.

4.6.1 Least Recently Used (LRU) Replacement policy:

Since program usually stay in localized areas for reasonable periods of time, it can be assumed that there is a high probability that blocks which have been referenced recently will also be referenced in the near future. Therefore, when a block is to be overwritten, it is a good decision to overwrite the one that has gone for longest time without being referenced. This is defined as

the least recently used (LRU) block. Keeping track of LRU block must be done as computation proceeds.

Consider a specific example of a four-block set. It is required to track the LRU block of this four-block set. A 2-bit counter may be used for each block. When a hit occurs, that is, when a read request is received for a word that is in the cache, the counter of the block that is referenced is set to 0. All counters which values originally lower than the referenced one are incremented by 1 and all other counters remain unchanged. When a miss occurs, that is, when a read request is received for a word and the word is not present in the cache, we have to bring the block to cache.

There are two possibilities in case of a miss:

If the set is not full, the counter associated with the new block loaded from the main memory is set to 0, and the values of all other counters are incremented by 1. If the set is full and a miss occurs, the block with the counter value 3 is removed, and the new block is put in its place. The counter value is set to zero. The other three block counters are incremented by 1. It is easy to verify that the counter values of occupied blocks are always distinct. Also it is trivial that highest counter value indicates least recently used block.

4.6.2 First in First out (FIFO) replacement policy:

A reasonable rule may be to remove the oldest from a full set when a new block must be brought in. While using this technique, ~~no updating is required when a hit occurs~~. When a miss occurs and the set is not full, the new block is put into an empty block and the counter values of the occupied block will be increment by one. When a miss occurs and the set is full, the block with highest counter value is replaced by new block and counter is set to 0, counter value of all other blocks of that set is incremented by 1. The overhead of the policy is less, since no updating is required during hit.

4.6.3 Random replacement policy:

The simplest algorithm is to choose the block to be overwritten at random. Interestingly enough, this simple algorithm has been found to be very effective in practice.

4.7 Main Memory

The main working principle of digital computer is *Von-Neumann* stored program principle. First of all we have to keep all the information in some storage, mainly known as main memory, and CPU interacts with the main memory only. Therefore, memory management is an important issue while designing a computer system.

On the other-hand, everything cannot be implemented in hardware, otherwise the cost of system will be very high. Therefore some of the tasks are performed by software program.

Replacement policy (Algorithm)

59

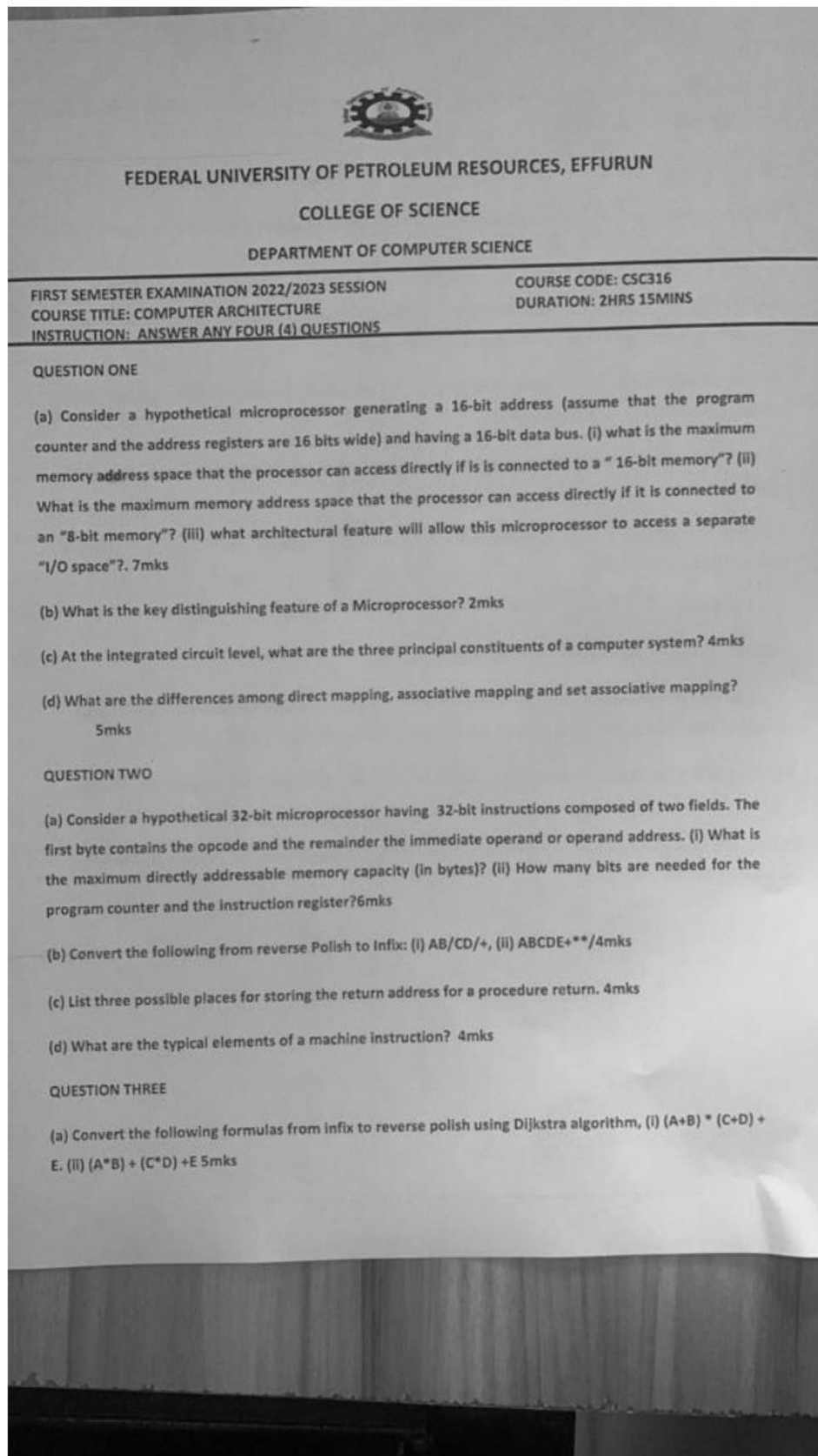
LRU Replacement Policy:

FIFO " ")
Random " "

Paper 3 of 3 — CSC 316 PAST QUESTION

Level 300 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



(b) Explain the cache memory principle and diagrammatically illustrate the relationship between the cache and main memory. 4mks

(c) If an instruction contains four addresses, what might be the purpose of each address? 5mks

(d) Define a cache memory and give reasons why we cannot have a big volume of cache memory. 4mks

QUESTION FOUR

(a) A computer is a complex system; contemporary computer contains millions of elementary electronic components. How, then can one clearly describe them? 6mks

(b) Find the following differences using twos complement arithmetic (i) $111000 - 110011$, (ii) $11001100 - 101110$ (iii) $11000011 - 11101000$. 2mks

(c) Justify the assertion that a 32-bit instruction is probably much less than twice as useful as a 16-bit instruction. 4mks

(d) State the various addressing modes with their algorithms, principal advantages and principal disadvantages of each. 6mks

QUESTION FIVE

(a) Define computer architecture and state its attributes. 4mks

(b) Show how the following floating point addition is performed (where significands are truncated to four decimal digits): (a) $0.5566 \times 10^3 + 0.7777 \times 10^3$ (b) $0.3344 \times 10^2 + 0.8877 \times 10^{-1}$. 4mks

(c) Using Dijkstra algorithm convert the expression $A + B * C (D + E) * F$ from infix to a Postfix notation. 5mks

(d) List and briefly explain two common ways of generating the condition to be tested in a conditional branch instruction. 5mks